DEEPSEED — 30-Day AI Mastery Roadmap

From Bootcamp Graduate to Domain Expert

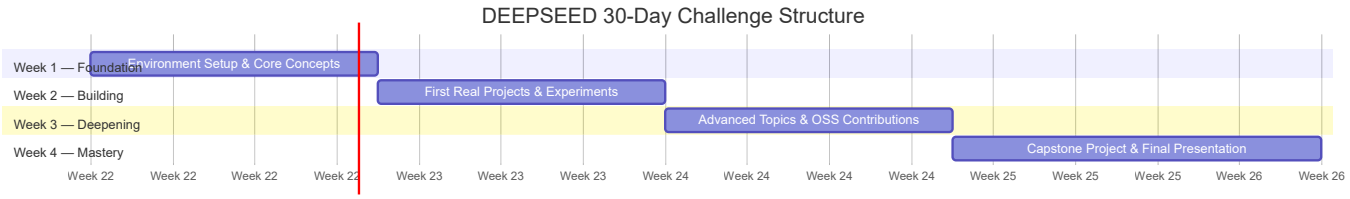
Written by: Fonyuy Gita
Community: DEEPSEED AI Community
Challenge Duration: 30 Days | **Presentations:** Every Sunday
Progress Tracking: GitHub (daily commits required)

> *"We don't just learn AI — we build with it, break it, secure it, and ship it. DEEPSEED is where bootcamp graduates become builders."*

How to Use This Roadmap

Each candidate picks **one field** from the eleven listed below and commits to it for 30 days. The 30 days are structured into four weekly phases: **Foundation** → **Building** → **Deepening** → **Mastery**. Every Sunday is a community presentation — not just a progress check, but a real demo of what you built that week.

Progress is tracked publicly on GitHub. Every day you should make at least one meaningful commit — whether it is a working script, notes in a `learnings.md`, a failed experiment with notes on why it failed, or a contribution to an open-source project. The commit history becomes your portfolio.



GitHub Progress Tracking — Setup (Day 0, Before You Start)

Before picking your field and diving in, every DEEPSEED member must set up their tracking repository. This is non-negotiable. Your GitHub is your proof of work.

Step 1 — Create your repo:

```
# Name it: deepseed-30days-<your-field>
# Example: deepseed-30days-llm-engineering
git init deepseed-30days-llm-engineering
cd deepseed-30days-llm-engineering
```

Step 2 — Create your folder structure on Day 0:

```
deepseed-30days-<field>/
├── README.md              # Your goals, field, and daily log table
├── week1/
│   ├── day01/
│   │   ├── code/          # Any scripts or notebooks you write
│   │   └── learnings.md   # What you learned, what confused you, links
│   └── ...
├── week2/ ...
├── week3/ ...
├── week4/
│   └── capstone/          # Your final project lives here
├── resources.md           # Books, papers, videos you used
└── presentation-slides/   # Your Sunday slide decks or demo links
```

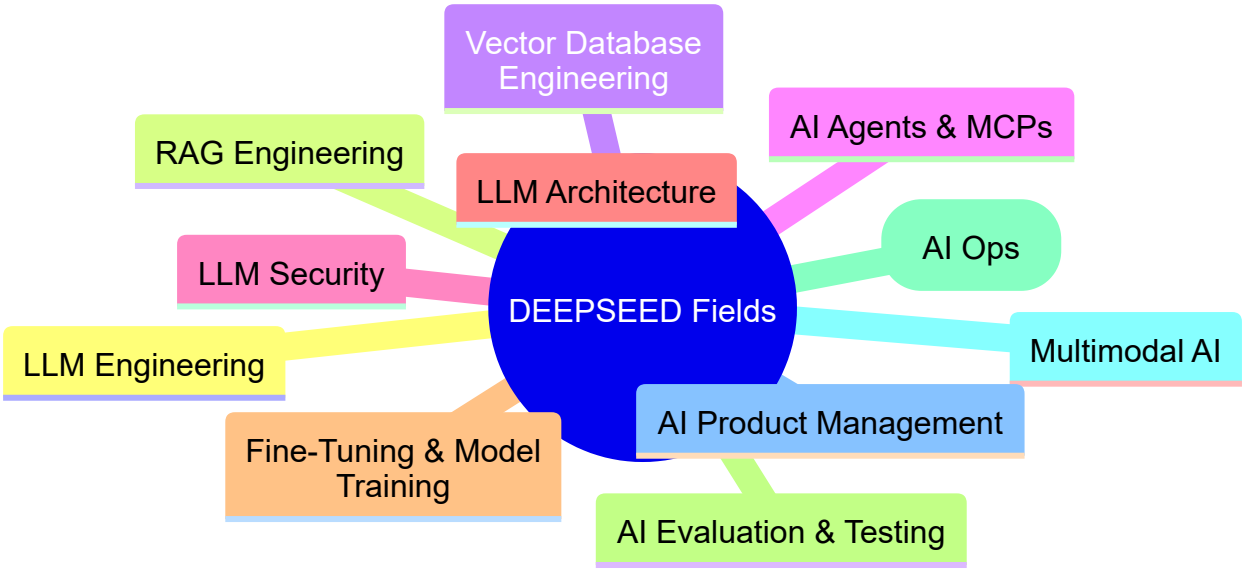
Step 3 — Daily commit rule:

```
# Every evening before bed, commit your day
git add .
git commit -m "Day 05 — Built basic RAG pipeline with Chroma, hit chunking issues"
git push origin main
```

Your `README.md` should have a progress table that you update daily:

Day	Date	Topic	Status	Key Output
01	...	Environment Setup	✅	Repo created, tools installed
02	...	Core Concept Study	✅	Notes in `week1/day02/learnings.md`
...				

The Eleven Fields

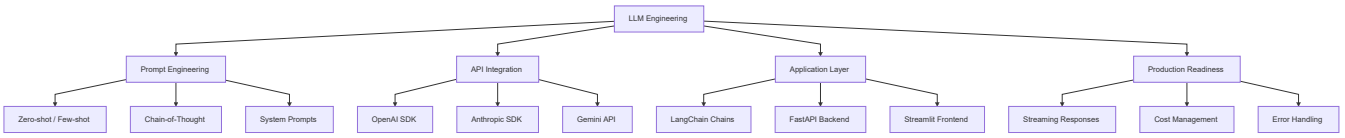


Pick **one**. Go deep. The community will hold you to it.

Field 1 — LLM Engineering

What it is: You are the builder who turns raw language models into real applications. You understand how to prompt well, how to chain calls intelligently, how to stream responses, manage costs, and ship production-ready AI features using APIs from OpenAI, Anthropic, Google, and others.

Core Tools: OpenAI Platform, Anthropic Claude API, Gemini, LangChain, FastAPI, Streamlit



Week 1 — Foundation (Days 1–7)

Day 1 — Environment Setup & API Access
Install Python 3.11+, create a virtual environment, and install `openai`, `anthropic`, `langchain`, `fastapi`, `streamlit`, and `python-dotenv`. Create accounts on OpenAI and Anthropic, generate API keys, and store them in a `.env` file. Write your first "Hello World" script that calls both the OpenAI and Claude APIs and prints the response. Commit this to GitHub.

Day 2 — Prompt Engineering Fundamentals
Study the anatomy of a prompt: system message, user message, assistant prefill. Practice zero-shot prompting (ask a question directly), few-shot prompting (give examples before your question), and chain-of-thought (ask the model to "think step by step"). Try the same task with all three approaches and compare output quality. Document your findings in `learnings.md`.

Day 3 — Tokens, Context Windows & Costs
Learn what tokens are and how they map to words. Use the OpenAI Tokenizer and `tiktoken` library to count tokens in your prompts. Understand context window limits for GPT-4o (128k), Claude Sonnet (200k), and Gemini (1M). Build a small script that estimates cost per API call before sending it. This is critical skill — engineers who ignore costs get fired.

Day 4 — Streaming & Async Calls
Implement streaming so your app shows text as it generates rather than waiting. Use `stream=True` in OpenAI and `stream=True` in Anthropic. Then convert your calls to async using Python's `asyncio` so you can handle multiple requests simultaneously. Build a small demo that streams a story word by word into a Streamlit UI.

Day 5 — LangChain Basics
LangChain is the glue layer between models and apps. Learn `ChatOpenAI`, `PromptTemplate`, `LLMChain`, and `RunnableSequence`. Build a chain that takes a user topic, generates a blog title, then generates an intro paragraph for it — two chained LLM calls. Understand why chaining matters for complex workflows.

Day 6 — Building a Streamlit App
Build your first full LLM application: a conversational chatbot with a Streamlit UI. It should maintain conversation history (multi-turn), have a system prompt you can customize, and show token usage per message. Deploy it locally and take a screenshot for your GitHub.

Day 7 — Sunday Presentation (Week 1)
Present your chatbot demo to the DEEPSEED community. Explain: what APIs you used, how prompting works, what you learned about costs and streaming. Show your GitHub commits from the week. Q&A with the community.

Week 2 — Building (Days 8–14)

Day 8 — FastAPI Backend for LLMs
Build a REST API that wraps your LLM calls using FastAPI. Create endpoints: `POST /chat` (sends a message, returns response), `POST /chat/stream` (streams the response), and `GET /models` (lists available models). Test with Postman or `curl`. This is how real production AI services are built.

Day 9 — Multi-Model Routing
Build a router that picks the right model for the job: use a cheap fast model (GPT-4o-mini) for short tasks, a powerful model (Claude Sonnet) for complex reasoning, and Gemini Flash for document-heavy tasks. The router should decide automatically based on prompt length and task type. This is a real engineering problem used in production.

Day 10 — Structured Outputs
Make LLMs return structured JSON reliably using Pydantic models and OpenAI's `response_format={"type": "json_object"}` or function calling. Build an information extractor that reads an unstructured paragraph about a person and returns a clean `{name, age, job, city}` JSON. Reliability of structured outputs is critical for downstream systems.

Day 11 — Function Calling / Tool Use
Teach the LLM to call external functions. Define tools like `get_weather(city)` and `search_web(query)` and let the model decide when to call them based on the user's message. Implement the full loop: model requests tool → you call function → return result to model → model answers. This is the foundation of AI agents.

Day 12 — Conversation Memory
Study different memory strategies: full conversation history (simple but expensive), sliding window (last N messages), and summary memory (LLM summarizes older context). Implement all three and compare cost vs quality. Add summary memory to your chatbot from Day 6.

Day 13 — Error Handling & Retries
Production LLM apps fail constantly — rate limits, timeouts, model errors. Implement exponential backoff with `tenacity`, handle rate limit errors gracefully, add fallback models (if OpenAI fails, try Anthropic), and log all errors. A brittle LLM app is worthless in production.

Day 14 — Sunday Presentation (Week 2)
Demo your FastAPI backend with multi-model routing and function calling. Show the router making intelligent model choices. Walk through your error handling code. The community should be able to ask questions about your architecture decisions.

Week 3 — Deepening (Days 15–21)

Day 15 — LangChain Expression Language (LCEL)
LCEL is the modern way to build LangChain pipelines using the pipe operator (`|`). Learn Runnables, parallel execution (`RunnableParallel`), and conditional branching (`RunnableBranch`). Refactor your earlier chains into LCEL and notice how much cleaner the code becomes.

Day 16 — Prompt Management & Templates
Large projects have dozens of prompts. Learn to manage them: store prompts in `.yaml` or `.json` files, version them in Git, load them dynamically. Explore LangSmith's prompt hub for sharing prompts. Build a prompt registry class that loads, versions, and logs every prompt used in your app.

Day 17 — LLM Caching
Identical prompts shouldn't cost money twice. Implement caching using LangChain's `InMemoryCache` and `SQLiteCache`. For a production scenario, use Redis as a cache backend. Build a benchmark showing how much time and money caching saves on repeated queries.

Day 18 — Open Source Contribution Day
Pick one issue from the LangChain GitHub repo (`python-langchain/langchain`) labeled `good first issue`. Read the codebase, understand the issue, write a fix, and submit a PR. Even if it doesn't get merged, the process teaches you how production LLM frameworks are built.

Day 19 — Advanced Prompt Patterns
Study and implement: ReAct (Reason + Act loops), Self-Consistency (run prompt N times, take majority answer), and Tree-of-Thought (explore multiple reasoning paths). These patterns push LLM accuracy significantly beyond naive prompting.

Day 20 — Cost Optimization & Monitoring
Build a cost dashboard using LangSmith or custom logging: track cost per user, per endpoint, per model. Implement prompt compression (remove unnecessary words before sending), context pruning (drop least relevant messages), and batching (combine multiple requests). Real companies care deeply about LLM costs.

Day 21 — Sunday Presentation (Week 3)
Present your cost optimization work. Show the before/after cost comparison. Discuss the open-source contribution you made. The community votes on the most impactful optimization of the week.

Week 4 — Mastery & Capstone (Days 22–30)

Days 22–27 — Capstone Project
Build a production-grade LLM application that combines everything: multi-model routing, function calling, structured outputs, memory, caching, error handling, and a FastAPI backend with Streamlit frontend. Suggested capstone ideas: an AI research assistant that searches the web and summarizes findings, a code review bot that reads a GitHub PR and gives structured feedback, or a customer support bot with escalation logic.

Day 28 — Documentation & README
Write thorough documentation for your capstone: architecture diagram in the README, setup instructions, API docs with examples, and a cost estimate for running it at scale. Your README is your business card.

Day 29 — Polish & Deploy
Deploy your app to a cloud platform (Railway, Render, or AWS Lambda). Add basic authentication to your API. Record a 3-minute demo video and link it from your GitHub README.

Day 30 — Final Presentation
Full demo to the DEEPSEED community. Present the problem, the architecture, live demo, lessons learned, and what you would do differently. This is your portfolio piece.

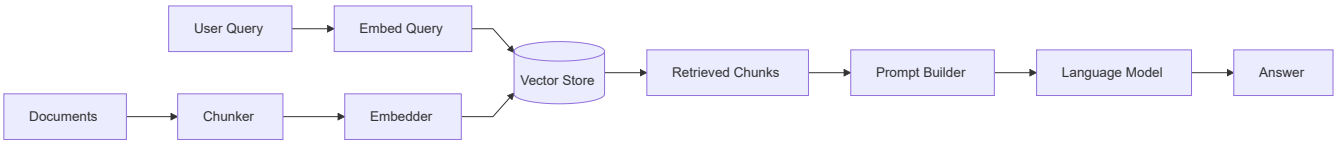
Top Open-Source Projects to Contribute To

The best way to go from user to expert is to read and contribute to the codebases that power the field. For LLM Engineering, start with **LangChain** (`langchain-ai/langchain`) — it is the most active LLM framework with thousands of issues and a welcoming maintainer team. Next, look at **LiteLLM** (`BerriAI/litellm`), a unified API that wraps 100+ LLM providers — excellent for understanding multi-model routing. **Instructor** (`jxnl/instructor`) is a beautiful library for structured outputs and is small enough that a newcomer can understand the whole codebase. Finally, **Semantic Kernel** (`microsoft/semantic-kernel`) is Microsoft's enterprise LLM SDK — contributing here has strong resume value.

Field 2 — RAG Engineering

What it is: RAG (Retrieval-Augmented Generation) is the technique of connecting an LLM to an external knowledge base so it can answer questions about documents, databases, or live data it was never trained on. You are the engineer who makes LLMs actually know things.

Core Tools: LlamaIndex, LangChain, Pinecone, Chroma, pgVector, Qdrant



Week 1 — Foundation (Days 1–7)

Day 1 — Setup & Core Concepts
Install `llama-index`, `langchain`, `chromadb`, `sentence-transformers`, and `openai`. Understand the three steps of RAG at a conceptual level: indexing (reading and embedding documents), retrieval (finding relevant chunks for a query), and generation (using retrieved context to answer). Draw this pipeline on paper before touching code.

Day 2 — Text Chunking Strategies
Chunking is where most RAG systems fail or succeed. Study fixed-size chunking (split every 512 tokens), recursive chunking (split by paragraphs, then sentences, then words), and semantic chunking (split at topic boundaries). Implement all three on a long PDF and compare the quality of chunks. Bad chunking = bad RAG, no matter how good your model is.

Day 3 — Embeddings Deep Dive
Embeddings convert text into numerical vectors that capture meaning. Learn what a vector is geometrically (a point in high-dimensional space). Use `text-embedding-3-small` from OpenAI and `all-MiniLM-L6-v2` from HuggingFace to embed the same sentences. Compute cosine similarity between pairs and notice that similar meaning = similar vectors. This intuition is everything in RAG.

Day 4 — Your First RAG Pipeline
Build a basic RAG pipeline from scratch (no frameworks): load a PDF → chunk it → embed each chunk → store in a list → embed a question → find top-3 similar chunks using cosine similarity → stuff them into a prompt → call the LLM → return answer. No Chroma, no LlamaIndex yet — raw code, so you understand what the frameworks are doing.

Day 5 — ChromaDB Integration
Refactor Day 4's pipeline using ChromaDB as the vector store. ChromaDB runs in-memory locally and is perfect for development. Store your chunks with metadata (source filename, page number, chunk index). Build a retrieval function that filters by metadata (e.g., "only search in chapter-3.pdf").

Day 6 — LlamaIndex Basics
LlamaIndex is the leading RAG framework. Learn its core abstractions: `SimpleDirectoryReader` (loads files), `VectorStoreIndex` (builds index), `QueryEngine` (runs RAG queries). Build a Q&A system over a folder of markdown notes in under 30 lines of code. Marvel at the abstraction, then go read the source to understand what it's doing.

Day 7 — Sunday Presentation (Week 1)
Demo your raw RAG pipeline vs LlamaIndex version. Show the difference in code length and explain the trade-offs. Walk through a chunking comparison. Explain what cosine similarity means in plain English.

Week 2 — Building (Days 8–14)

Day 8 — Advanced Retrieval Strategies
Basic top-k retrieval is just the start. Implement: MMR (Maximum Marginal Relevance — diverse results), hybrid search (vector + keyword BM25 combined), and parent-document retrieval (embed small chunks, retrieve large parent chunks). Each strategy solves a different failure mode of naive retrieval.

Day 9 — Pinecone Production Setup
Move from ChromaDB (local) to Pinecone (cloud, production-grade). Create an index with the right dimensions and metric. Understand namespaces (separate collections within one index) and metadata filtering. Build a multi-tenant RAG system where different users get different knowledge bases using namespaces.

Day 10 — Reranking
After retrieving top-20 chunks, use a reranker model (Cohere Rerank or `cross-encoder/ms-marco-MiniLM-L-6-v2`) to reorder them by true relevance to the query. Retrieval finds candidates; reranking picks the winners. This alone can improve RAG accuracy by 15–30%. Measure the difference on your test set.

Day 11 — Query Transformation
Users ask bad questions. Fix this before retrieval. Implement: HyDE (Hypothetical Document Embeddings — generate a fake answer to the question, then search for documents similar to that answer), query expansion (rewrite the question 3 ways, search with all 3), and step-back prompting (abstract the question to a higher level before searching). These techniques dramatically improve retrieval quality.

Day 12 — Qdrant Deep Dive
Qdrant is a high-performance vector database with powerful filtering. Set it up with Docker. Learn payload filtering (filter by metadata at query time, not after), sparse+dense vectors for hybrid search, and collections. Build a product search demo using Qdrant where users filter by category and price range while doing semantic search.

Day 13 — Evaluation Basics
How do you know your RAG is working? Build a small evaluation set: 20 questions with expected answers. Measure: faithfulness (does the answer come from retrieved docs?), answer relevance (does the answer address the question?), and context precision (are retrieved chunks actually relevant?). Use `ragas` library for this.

Day 14 — Sunday Presentation (Week 2)
Demo query transformation in action. Show before/after retrieval quality with reranking. Present your evaluation scores and what they mean.

Week 3 — Deepening (Days 15–21)

Day 15 — pgVector: RAG in Postgres
pgVector adds vector search to PostgreSQL, the world's most popular database. This matters because most companies already have Postgres. Install pgVector with Docker, store embeddings alongside regular data, and build queries that combine SQL filters with vector similarity search. This is the pragmatic choice for many production systems.

Day 16 — Multimodal RAG
Build a RAG system that handles not just text but images. Use GPT-4o's vision capability to index images by generating text descriptions, then embed those descriptions. When a user asks about something visual, retrieve the image description and reference the image. This is the future of enterprise knowledge bases.

Day 17 — Agentic RAG
Static RAG answers one question at a time. Agentic RAG uses an LLM to decide *how* to search, *whether* to search multiple times, and *how* to combine results. Build a research agent using LangGraph that: decides if the question needs retrieval, searches, reads results, decides if it needs more information, searches again if needed, then synthesizes a final answer.

Day 18 — Open Source Contribution Day
Contribute to **LlamaIndex** (`run-llama/llama_index`) — look for issues labeled `good first issue`. Alternatively, contribute to **Chroma** (`chroma-core/chroma`) or **Ragas** (`explodinggradients/ragas`). The Ragas project especially welcomes contributions to its evaluation metrics.

Day 19 — Caching & Optimization
Cache frequent queries so you don't re-embed and re-search the same question repeatedly. Use `GPTCache` or build a simple Redis cache keyed by query embedding. Also study approximate nearest neighbor algorithms (HNSW, IVF) that make vector search fast at scale. Understand why exact search is O(n) and why HNSW is O(log n).

Day 20 — RAG for Long Documents
Books, legal contracts, and research papers are too long for simple RAG. Study: hierarchical indexing (index summaries, then drill into chapters), map-reduce over chunks, and sliding window retrieval. Build a system that can answer questions across a full 300-page PDF book.

Day 21 — Sunday Presentation (Week 3)
Present your agentic RAG system. Show it deciding to do multiple retrieval rounds. Compare its answers against basic RAG on hard questions. Live demo always wins.

Week 4 — Capstone (Days 22–30)

Build a complete RAG-powered knowledge assistant over a domain you care about — your field's documentation, a company's internal docs, or a large book collection. The system must handle multi-turn conversations, use Pinecone or Qdrant as the vector store, implement reranking, evaluate itself on a 30-question test set, and have a clean FastAPI backend. Deploy it and record a demo video.

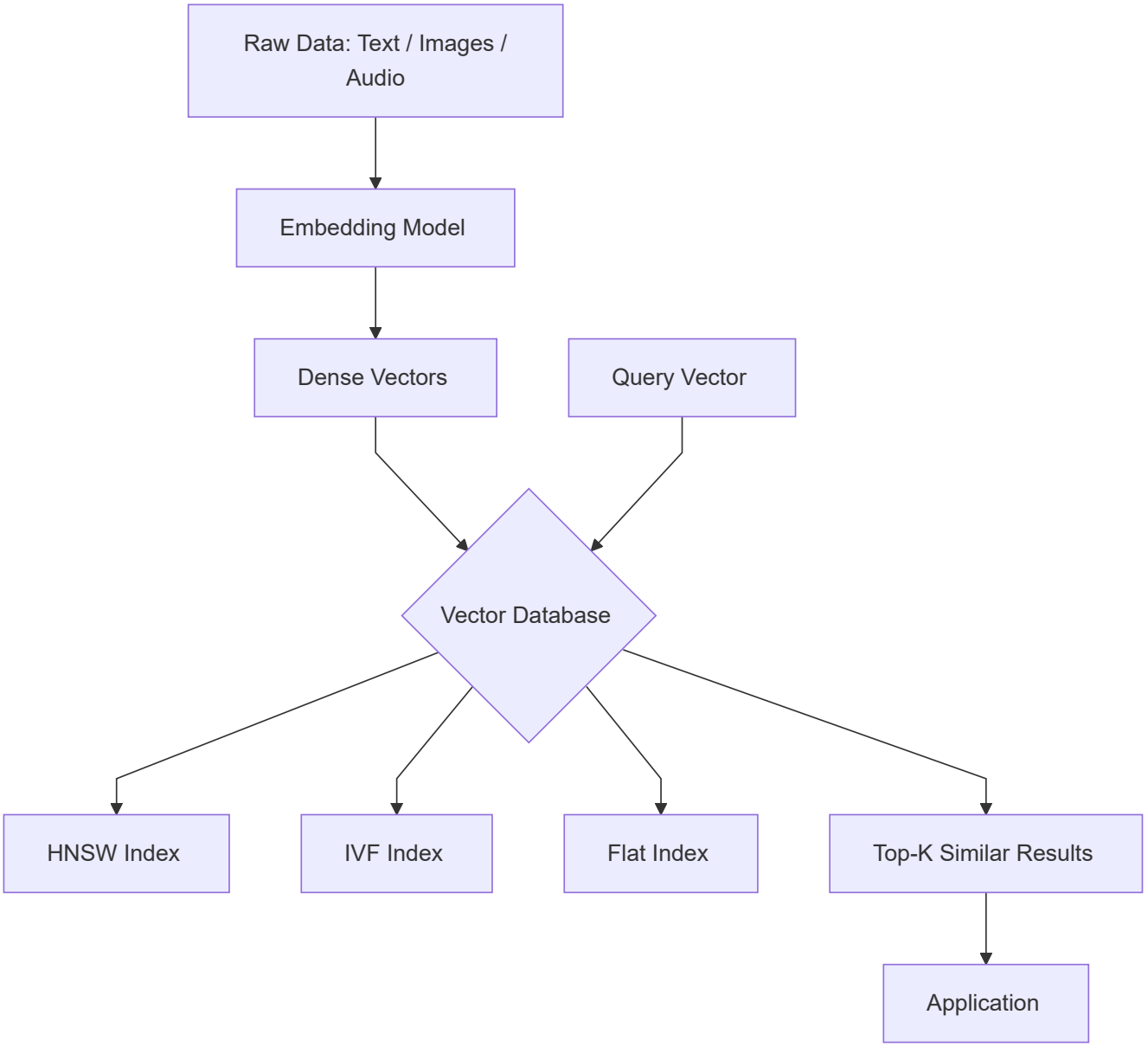
Top Open-Source Projects to Contribute To

LlamaIndex (`run-llama/llama_index`) is the home of RAG research in code form — new retrieval strategies appear here weekly and there are always open issues. **Ragas** (`explodinggradients/ragas`) is the standard RAG evaluation library and actively needs more metric implementations. **Chroma** (`chroma-core/chroma`) is the most popular local vector store for development and has excellent contributor documentation. **Haystack** (`deepset-ai/haystack`) is an enterprise RAG framework from deepset, with great architecture for learning production patterns.

Field 3 — Vector Database Engineering

What it is: You specialize in the storage, indexing, and retrieval of high-dimensional vector embeddings. You understand the mathematics of similarity search, can scale to billions of vectors, and design systems that make semantic search fast and accurate.

Core Tools: Pinecone, Qdrant, Weaviate, Milvus, Chroma



Week 1 — Foundation (Days 1–7)

Day 1 — The Mathematics of Similarity
Before touching a database, understand the math. What is a vector? (A list of floats representing a point in N-dimensional space.) What is cosine similarity, Euclidean distance, and dot product? When do you use each metric? Build a Python script that computes all three for pairs of sentence embeddings and plots the relationships. This day is all math and exploration — no databases yet.

Day 2 — Embedding Models Survey
Different tasks need different embedding models. Study: OpenAI `text-embedding-3-large` (best quality, paid), `BAAI/bge-m3` (multilingual, free), `nomic-embed-text` (long context, free), and CLIP (multimodal text+image). Benchmark them: embed 100 sentences, build nearest-neighbor graphs, measure how well semantically similar sentences cluster together. Pick your benchmark dataset from MTEB.

Day 3 — Approximate Nearest Neighbor Algorithms
Exact nearest neighbor search is O(n) — too slow at scale. Study HNSW (Hierarchical Navigable Small World — the gold standard), IVF (Inverted File Index — cluster first, search subset), and Product Quantization (compress vectors to save memory). You don't need to implement them, but you must understand the trade-offs: speed vs accuracy vs memory. This knowledge determines how you configure production databases.

Day 4 — ChromaDB & Local Development
Start practical. Chroma is the go-to local vector store. Build a personal semantic search engine over your own notes and documents: embed everything, store in Chroma, query with natural language. Add metadata filtering. Understand Chroma's persistence mode vs in-memory mode. Write a benchmark: search over 10k documents and measure query latency.

Day 5 — Pinecone: Cloud-Native Vector Search
Pinecone is serverless, scales to billions of vectors, and is what many startups use in production. Create a Pinecone account, create an index (choose dimension to match your embedding model), upsert 50k vectors with metadata, and build search queries with metadata filters. Understand pods vs serverless pricing — this knowledge prevents budget disasters.

Day 6 — Qdrant: The Engineer's Choice
Qdrant is open-source, self-hostable, and has the richest filtering system. Run it with Docker: `docker run -p 6333:6333 qdrant/qdrant`. Learn: collections, payload (metadata), named vectors (store multiple vector types per record), sparse vectors (for hybrid search). Build a product search system: each product has a text description vector AND an image vector, users can search with either.

Day 7 — Sunday Presentation (Week 1)
Explain HNSW in plain English using a diagram. Demo your personal semantic search engine. Show benchmark results across different databases and configurations.

Week 2 — Building (Days 8–14)

Day 8 — Weaviate: GraphQL + Vectors
Weaviate is unique in combining vector search with a GraphQL API and built-in module system (it can call embedding models internally). Set it up with Docker, define a schema, and import data. Use GraphQL to run hybrid queries (BM25 + vector). Weaviate is popular in enterprise environments — knowing it differentiates you.

Day 9 — Milvus: Billion-Scale Search
Milvus is the database that handles truly massive scale — billions of vectors, distributed across clusters. Set it up with `milvus-lite` (runs locally). Learn collections, partitions, and indexes. Understand when you need Milvus vs when Qdrant or Pinecone are sufficient. Build a benchmark comparing Milvus vs Qdrant at 1M vectors.

Day 10 — Hybrid Search: Dense + Sparse
Pure semantic search misses exact keyword matches. Pure keyword search misses meaning. Hybrid search combines both. Implement BM25 (keyword) + dense vector search with Reciprocal Rank Fusion (RRF) to merge results. Qdrant, Weaviate, and Elasticsearch natively support this. Measure: on a benchmark dataset, hybrid beats either alone by 10–20%.

Day 11 — Filtered Vector Search
When you have 10M product vectors and a user filters by `category="electronics" AND price<100`, you can't search all 10M vectors. Learn pre-filtering (filter first, then search subset), post-filtering (search all, then filter — loses top-k accuracy), and ACORN (Qdrant's approach that integrates filters into the graph traversal). This is a hard distributed systems problem.

Day 12 — Indexing Strategies for Production
When should you use HNSW vs IVF? How do you choose `ef`, `m`, and `ef_construction` parameters for HNSW? What is the trade-off between index build time, memory usage, and query speed? Build a systematic experiment: vary HNSW parameters and measure recall@10 (fraction of true top-10 results returned) vs queries-per-second.

Day 13 — Multi-Tenancy in Vector Databases
SaaS products need to isolate data per customer. Learn the three patterns: one collection per tenant (expensive but fully isolated), namespaces/partitions (efficient but shared infrastructure), and metadata filtering per query (simplest but full scan risk). Implement all three in Qdrant and compare performance at 100 tenants, each with 10k vectors.

Day 14 — Sunday Presentation (Week 2)
Present your hybrid search results with metrics. Show the multi-tenancy benchmark. Explain the HNSW parameter experiment and what the recall vs speed trade-off means for different use cases.

Week 3 — Deepening (Days 15–21)

Day 15 — Embedding Updates & Versioning
Embedding models improve over time. When OpenAI releases a new model, do you re-embed your entire database? Learn: two-tower models (keep old and new in parallel), incremental reindexing, and ANN index updates vs rebuilds. This is a real operational challenge at scale.

Day 16 — Vector Quantization for Memory Efficiency
Storing 1B 1536-dimensional float32 vectors costs ~6TB of RAM. That's impossible. Learn Product Quantization (PQ), Scalar Quantization (SQ), and Binary Quantization — techniques that compress vectors 4x–64x with minimal accuracy loss. Implement SQ in Qdrant and measure the memory/recall trade-off.

Day 17 — Building a Semantic Search API
Build a production-grade semantic search service: FastAPI backend with Qdrant, support for multiple collections, batch upsert endpoint, search endpoint with filters and pagination, health check endpoint, and OpenTelemetry tracing. This is a deployable microservice.

Day 18 — Open Source Contribution Day
Contribute to **Qdrant** (`qdrant/qdrant`) — the Rust codebase is high quality and they label issues helpfully. Alternatively, contribute to **Chroma** (`chroma-core/chroma`) in Python, or **pgvector** (`pgvector/pgvector`) which is written in C. Even documentation improvements and benchmark additions are highly valued.

Day 19 — Monitoring Vector Databases in Production
What breaks in production? Index corruption, shard imbalances, slow queries, memory pressure. Set up Grafana + Prometheus dashboards for Qdrant (it exposes `/metrics` natively). Build alerts for: p99 query latency > 100ms, memory usage > 80%, and failed upserts. Operations matter as much as engineering.

Day 20 — Semantic Caching
Cache LLM answers by query similarity. If a user asks "What is the capital of France?" and someone already asked "What's France's capital?", you should return the cached answer. Build this with a vector database: embed every query, search for similar past queries (cosine > 0.95 = cache hit), return cached answer. This cuts LLM costs dramatically for popular queries.

Day 21 — Sunday Presentation (Week 3)
Present your semantic caching system with hit-rate metrics. Show the monitoring dashboard. Deep-dive on quantization results.

Week 4 — Capstone (Days 22–30)

Build a large-scale semantic search system: index at least 100k real documents (use the Wikipedia dataset or Common Crawl subset), implement hybrid search with reranking, build a multi-tenant API, add quantization, deploy with Docker Compose (Qdrant + FastAPI + Redis for semantic cache), and write a comprehensive benchmark report. Your capstone proves you can operate vector databases at real scale.

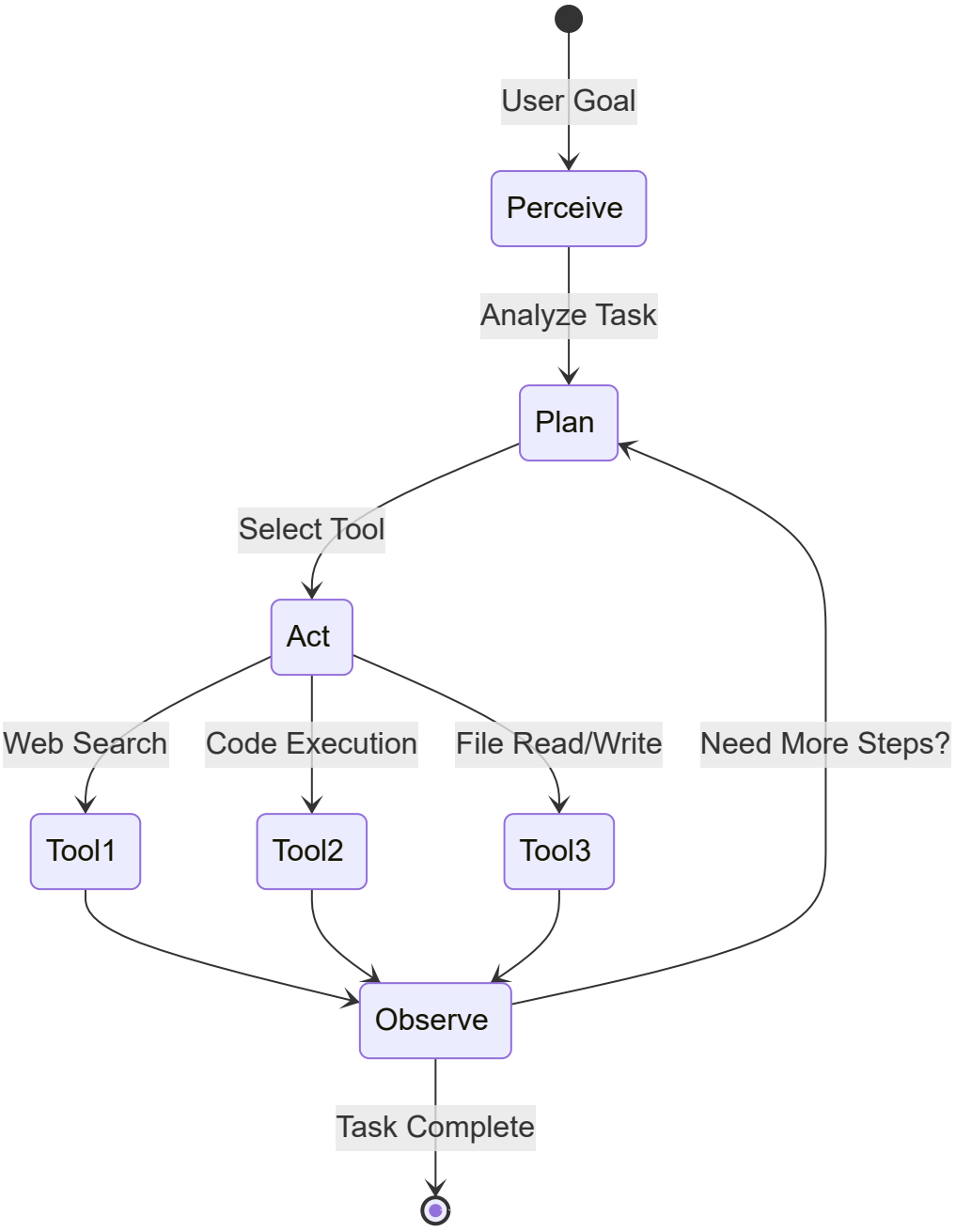
Top Open-Source Projects to Contribute To

Qdrant (`qdrant/qdrant`) is written in Rust and is one of the most well-engineered open-source vector databases — reading its codebase alone teaches you distributed systems. **Chroma** (`chroma-core/chroma`) is Python-native and has simpler internals, perfect for first contributions. **pgvector** (`pgvector/pgvector`) brings vector search to PostgreSQL and contributions here have enormous reach given Postgres's ubiquity. **Weaviate** (`weaviate/weaviate`) is Go-based and enterprise-focused with an active community. **FAISS** (`facebookresearch/faiss`) from Meta is the foundational library that most vector databases build on — contributing here means contributing to the base layer of the entire field.

Field 4 — AI Agents & MCP Engineering

What it is: You build autonomous AI systems that can plan, reason, use tools, and execute multi-step tasks without human intervention. Agents are the frontier of applied AI — they are where LLMs go from question-answering to actually doing things.

Core Tools: LangGraph, CrewAI, AutoGen, OpenAI Agents SDK, Vercel AI SDK, Google ADK, MCP (Model Context Protocol)



Week 1 — Foundation (Days 1–7)

Day 1 — What Makes an Agent an Agent?
Study the ReAct paper (Reason + Act) — this is the conceptual foundation of all modern agents. An agent is an LLM in a loop: it reasons about what to do, takes an action (calls a tool), observes the result, reasons again, and repeats until the task is done. Implement a bare-bones ReAct loop from scratch — no frameworks — using just the OpenAI API and a Python while loop. This is the most important day of the whole 30.

Day 2 — Tool Design & Function Calling
Agents are only as good as their tools. Learn to design tools with clear names, good descriptions (the LLM reads these to decide which to use), and strict input schemas. Build a toolkit: `search_web(query)`, `read_file(path)`, `write_file(path, content)`, `run_python(code)`, and `send_email(to, subject, body)`. The description quality determines how well agents select tools.

Day 3 — OpenAI Agents SDK
The OpenAI Agents SDK (formerly Swarm) is the simplest way to build production agents. Learn: `Agent` (defines an LLM with tools and instructions), `Runner` (executes the agent), handoffs (one agent passes control to another), and guardrails (validate inputs/outputs). Build a customer support agent that routes to specialized sub-agents (billing agent, technical agent, returns agent).

Day 4 — LangGraph Fundamentals
LangGraph treats an agent as a graph of nodes (LLM calls, tool calls, logic) connected by edges (conditional transitions). This is powerful because you can explicitly control flow. Learn: `StateGraph`, `add_node`, `add_edge`, `add_conditional_edges`, and `compile`. Build a simple research agent graph: search → summarize → fact-check → respond.

Day 5 — MCP: Model Context Protocol
MCP (from Anthropic) is the USB-C of AI tools — a standard protocol for connecting LLMs to external data sources and tools. Learn the MCP spec: servers expose tools and resources, clients (LLMs) discover and call them. Set up an MCP server using the `mcp` Python SDK. Build an MCP server that exposes your local filesystem to an AI assistant. This is the future of agent tool ecosystems.

Day 6 — Memory in Agents
Agents need memory to be useful across sessions. Study: working memory (the current context window), episodic memory (past conversations, stored as embeddings, retrieved when relevant), procedural memory (learned skills, stored as prompts), and semantic memory (facts about the world, stored in a knowledge base). Implement episodic memory for your agent using a vector store.

Day 7 — Sunday Presentation (Week 1)
Walk through your ReAct implementation line by line. Show the agent reasoning, picking tools, and failing gracefully when a tool doesn't work. The community should understand what an agent loop looks like in raw code.

Week 2 — Building (Days 8–14)

Day 8 — CrewAI: Multi-Agent Teams
CrewAI lets you define crews of agents with different roles and have them collaborate. Learn: `Agent` (with role, goal, backstory), `Task` (with description and expected output), and `Crew` (orchestrates agents in sequential or hierarchical order). Build a content creation crew: a researcher agent, a writer agent, and an editor agent working together on an article.

Day 9 — AutoGen: Conversational Multi-Agent
AutoGen from Microsoft takes a different approach — agents communicate through conversation. Learn `ConversableAgent`, group chats, and human-in-the-loop patterns where a human can intervene in an agent conversation. Build a coding assistant where a programmer agent writes code and a critic agent reviews it, going back and forth until the code passes tests.

Day 10 — Stateful Agents with LangGraph
LangGraph's true power is stateful, persistent workflows. Learn checkpointing (save agent state to a database so it can resume), time travel (replay from any previous state for debugging), and human-in-the-loop interruption (pause the agent and wait for human approval before continuing). These features are what make agents production-safe.

Day 11 — Agent Planning Strategies
Flat ReAct loops struggle with complex multi-step tasks. Study: Plan-and-Execute (generate a full plan upfront, then execute step by step), Hierarchical Planning (a planner agent breaks tasks into subtasks assigned to worker agents), and LATS (Language Agent Tree Search — explore multiple plans, pick the best). Implement Plan-and-Execute for a complex research task.

Day 12 — Building MCP Servers
Build production-quality MCP servers that expose real tools. Create an MCP server for: database queries (expose SQL as a tool), web search (wrap a search API), and code execution (run Python in a sandbox). Connect them to Claude Desktop or a custom agent. Understanding MCP server architecture puts you ahead of 95% of the field.

Day 13 — Agent Safety & Guardrails
Autonomous agents can do dangerous things. Implement: input guardrails (validate user intent before running), output guardrails (check agent responses for harmful content), action confirmation (ask for human approval before irreversible actions like sending emails or deleting files), rate limiting (prevent agent from making too many calls), and scope limiting (agent can only access predefined tools). Safety is what separates toy agents from production agents.

Day 14 — Sunday Presentation (Week 2)
Demo your multi-agent crew building content. Show LangGraph's checkpoint feature resuming an interrupted workflow. Walk through your safety guardrails and explain why each one matters.

Week 3 — Deepening (Days 15–21)

Day 15 — Google ADK (Agent Development Kit)
Google's ADK provides a unified framework for building agents that work with Gemini models and Google Cloud services. Learn its approach to tool definition, session management, and deployment. Build an agent using ADK and compare the developer experience with LangGraph and OpenAI Agents SDK. Understanding multiple frameworks makes you more versatile.

Day 16 — Vercel AI SDK for Agent UIs
Agents need UIs. Vercel AI SDK provides React hooks for streaming agent responses, tool call visualizations, and generative UIs (where the agent renders interactive UI components, not just text). Build a React app where an agent shows its reasoning steps, tool calls, and results in real time. This is how Vercel's v0 and similar products work.

Day 17 — Evaluation of Agents
How do you know if an agent is good? Agent evaluation is hard because the path to a correct answer matters, not just the answer. Study: task completion rate, tool call accuracy, number of steps to completion, and cost per task. Use `AgentEvals` or build custom evals. Create a benchmark of 20 tasks and evaluate three different agent architectures against it.

Day 18 — Open Source Contribution Day
Contribute to **LangGraph** (`langchain-ai/langgraph`) — it has active development and regularly needs documentation improvements, new node types, and bug fixes. **AutoGen** (`microsoft/autogen`) is Microsoft-backed with a large community. **CrewAI** (`crewAIInc/crewAI`) is smaller and more welcoming to newcomers. Even adding a tutorial to any of these repos is a meaningful contribution.

Day 19 — Agentic Code Execution
Code-executing agents are the most powerful class. Build an agent that can: write Python scripts to answer data questions, run them in a sandboxed environment (use `subprocess` with resource limits or a Docker container), read the output, fix errors, and retry. This is the foundation of products like Devin, GitHub Copilot Workspace, and Claude Code.

Day 20 — Long-Running Agents & Async Workflows
Some agent tasks take minutes or hours (e.g., research a company's entire website). Learn: async agent execution (don't block the user while the agent works), webhook callbacks (notify when done), job queues (Redis Queue or Celery to manage agent jobs), and progress streaming (send updates as the agent works). Build an async research agent with a job queue.

Day 21 — Sunday Presentation (Week 3)
Demo your code-executing agent solving a real data problem live. Show the async agent system with job queue. Present evaluation results across agent architectures.

Week 4 — Capstone (Days 22–30)

Build a multi-agent system that solves a real-world problem: a research assistant that autonomously plans, searches multiple sources, synthesizes findings, and produces a structured report; a software development agent that takes a GitHub issue, writes code, runs tests, and opens a PR; or a business intelligence agent that queries a database, creates charts, and emails a report. The system must use at least 2 agents, implement safety guardrails, have persistent memory, and include an evaluation suite.

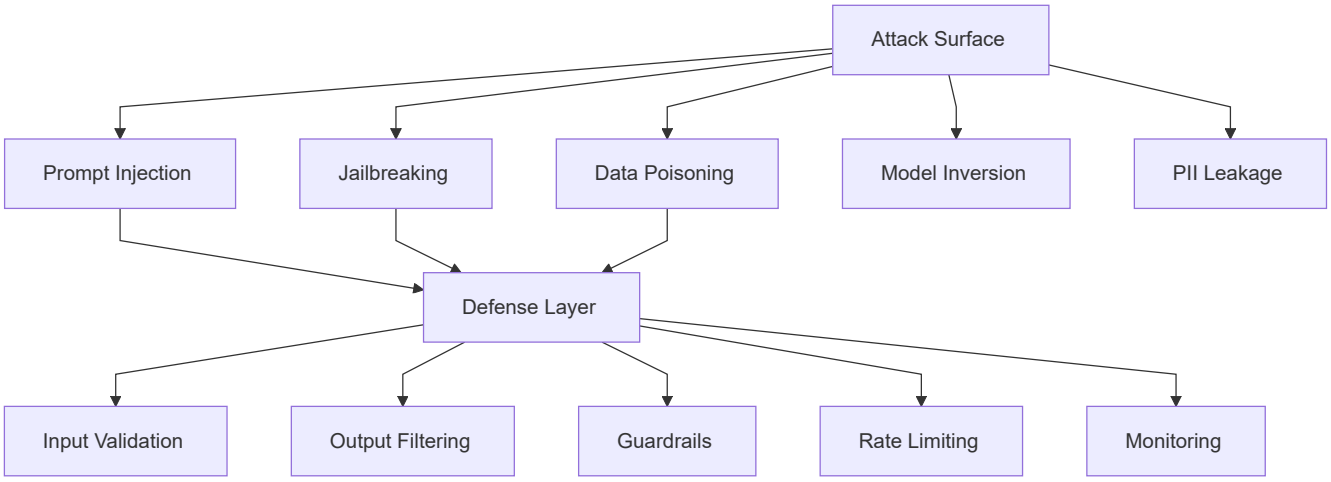
Top Open-Source Projects to Contribute To

LangGraph (`langchain-ai/langgraph`) is the most sophisticated agent framework and has the most educational codebase — reading how it implements checkpointing and time-travel teaches distributed systems. **AutoGen** (`microsoft/autogen`) is widely used in research and enterprise. **CrewAI** (`crewAIInc/crewAI`) is the friendliest to newcomers and has clear contribution guidelines. **OpenHands** (`All-Hands-AI/OpenHands`) — formerly OpenDevin — is an open-source version of Devin and is one of the most exciting agent projects to contribute to. **Agno** (`agno-agi/agno`) is a lightweight agent framework that is growing fast.

Field 5 — LLM Security

What it is: You protect AI systems from adversarial attacks, prevent misuse, and ensure that LLMs behave safely and reliably in production. This is one of the most critical and underserved specializations in the industry.

Core Tools: OWASP GenAI Security Project, Guardrails AI, Lakera, Promptfoo



Week 1 — Foundation (Days 1–7)

Day 1 — OWASP Top 10 for LLMs
The OWASP Top 10 for LLM Applications is your bible. Study all ten: Prompt Injection (#1), Insecure Output Handling, Training Data Poisoning, Model Denial of Service, Supply Chain Vulnerabilities, Sensitive Information Disclosure, Insecure Plugin Design, Excessive Agency, Overreliance, and Model Theft. Read the full OWASP document and create a summary with one example attack for each. This is the foundation.

Day 2 — Prompt Injection: Theory & Practice
Prompt injection is the SQL injection of AI — an attacker sneaks malicious instructions into user input that override the system prompt. Types: direct injection (user directly attacks the model), indirect injection (malicious content in a document the agent reads and acts on). Build a vulnerable LLM app, then attack it yourself with 10 different injection techniques. You cannot defend what you haven't attacked.

Day 3 — Jailbreaking Techniques
Study the major jailbreak families: DAN (Do Anything Now), role-playing exploits ("pretend you are an AI with no restrictions"), many-shot jailbreaking (using a long context of examples to condition behavior), and suffix attacks (adversarial text appended to a prompt that reliably jailbreaks the model). Use `garak` (the LLM vulnerability scanner) to probe a model for common weaknesses.

Day 4 — Guardrails AI
Guardrails AI wraps your LLM calls with validators that check both inputs and outputs. Learn: `Guard`, `Validator`, `OnFailAction`. Build guards for: detecting PII in outputs (names, emails, phone numbers), checking that responses are topically relevant (no off-topic answers), validating JSON structure, and detecting toxic language. Install `guardrails-ai` and build a fully guarded Q&A endpoint.

Day 5 — Lakera & Real-Time Protection
Lakera Guard is a production-grade API that classifies whether a prompt is a prompt injection attempt in real time. Integrate it into your FastAPI LLM endpoint: before every LLM call, send the user input to Lakera, and if the score is above a threshold, reject the request. Study Lakera's attack taxonomy and understand what signals it uses to detect attacks.

Day 6 — PII Detection & Redaction
LLMs memorize training data, and users sometimes input sensitive information. Build a PII detection layer using Microsoft Presidio (open-source) that: identifies names, emails, phone numbers, SSNs, credit card numbers, and medical record numbers, then redacts or pseudonymizes them before sending to the LLM, and

restores them in the response. This is required for HIPAA and GDPR compliance.

Day 7 — Sunday Presentation (Week 1)
Demonstrate a prompt injection attack live on a vulnerable system, then show how your defenses stop it. Walk through the OWASP Top 10 with examples. The community should leave understanding why LLM security is a serious engineering discipline, not just a policy concern.

Week 2 — Building (Days 8–14)

Day 8 — Promptfoo: Automated Red Teaming
Promptfoo lets you run automated test suites against your LLM application — including adversarial tests. Set up a `promptfooconfig.yaml` that tests your app against hundreds of jailbreak attempts, prompt injections, and policy violations automatically. Integrate this into a CI/CD pipeline so every deployment is automatically red-teamed. Security that isn't automated is security that gets skipped.

Day 9 — Secure System Prompt Design
The system prompt is your primary defense layer. Learn: prompt hardening (explicit instructions about what not to do), context isolation (separate system prompt from user input with clear delimiters), canary tokens (detect if the system prompt has been leaked), and instruction hierarchy (why instructions at the top of the system prompt are weaker than at the bottom). Redesign your chatbot's system prompt using all these techniques.

Day 10 — Data Poisoning & Supply Chain Attacks
If you fine-tune a model on poisoned data, the model can be backdoored — triggered to behave maliciously by a specific input pattern. Study the literature on backdoor attacks. For supply chain, understand: if you use a third-party LLM tool with a security vulnerability, your whole app is vulnerable. Audit every dependency in an LLM app stack for known CVEs.

Day 11 — Model Denial of Service
An attacker can send prompts designed to maximize compute: prompts with deeply nested instructions, requests for extremely long outputs, or adversarial inputs that trigger degenerate token generation loops. Build rate limiting (max N requests per user per minute), output length limits, and prompt complexity limits. Load test your endpoint to find the breaking point before attackers do.

Day 12 — Multimodal Attack Vectors
Images can contain invisible text (adversarial examples) that GPT-4o reads and acts on. Audio can contain ultrasonic commands inaudible to humans but processed by Whisper. Study these multimodal attack vectors. Build a content moderation pipeline for images submitted to an LLM app using the OpenAI moderation endpoint and Microsoft's Content Safety API.

Day 13 — Security Monitoring & Alerting
Production LLM apps need real-time security monitoring. Build a logging system that records every prompt and response, runs async security checks (PII detection, injection detection, policy violation checks), and triggers alerts on anomalies (sudden spike in rejected prompts = likely attack). Use LangSmith or build a custom dashboard with Grafana.

Day 14 — Sunday Presentation (Week 2)
Live demo of Promptfoo automated red-teaming against your app. Show the security dashboard catching an attack in real time. Discuss the multimodal attack vectors you researched.

Week 3 — Deepening (Days 15–21)

Day 15 — Agentic Security: A Different Problem
Agents that can execute code, call APIs, and delete files are catastrophically dangerous if compromised. Study: minimal tool exposure (agents should have only the tools they absolutely need), confirmation gates (human approval for irreversible actions), sandboxing (run agent actions in isolated environments), and confused deputy attacks (an agent is tricked into misusing a privileged tool). Write a threat model for a code-executing agent.

Day 16 — Privacy-Preserving LLM Applications
Build an LLM app that respects privacy by design: use on-premises models (Ollama) for sensitive queries, implement differential privacy for analytics, apply federated learning concepts, and never log raw user inputs. This is the architecture required for healthcare, legal, and financial LLM applications.

Day 17 — LLM Firewall Architecture
Design and build a complete LLM firewall: an API proxy that sits between your users and the LLM API, performing: input classification (intent detection), PII redaction, injection detection, output filtering, rate limiting, and audit logging. Every LLM request goes through the firewall. This is what enterprise AI security looks like.

Day 18 — Open Source Contribution Day
Contribute to **Guardrails AI** (`guardrails-ai/guardrails`) — they actively need new validators and the codebase is well-structured. **Garak** (`leondz/garak`), the LLM vulnerability scanner, needs new probes for emerging attack types. **LLM Guard** (`protectai/llm-guard`) is a lightweight alternative that also needs contributors. Any new attack detection method you implement is a genuine contribution to the field's security.

Day 19 — Red Team Exercise
Team up with another DEEPSEED member. Spend 3 hours attacking each other's LLM applications and 3 hours defending. Document every successful attack and every failed attack. Write a post-mortem with specific fixes. Real security knowledge comes from adversarial practice, not just reading.

Day 20 — Compliance & Governance
Study AI governance frameworks: EU AI Act (risk classification, prohibited practices), NIST AI RMF (identify, govern, map, measure, manage risks), and GDPR implications for LLM training data. Build a compliance checklist for deploying an LLM application in a regulated industry (healthcare, finance, or legal). This skill is increasingly required for senior AI engineering roles.

Day 21 — Sunday Presentation (Week 3)
Present your LLM firewall architecture with a live demo of attacks being blocked. Share the red team exercise findings. Discuss one real-world LLM security incident from the news and how better engineering could have prevented it.

Week 4 — Capstone (Days 22–30)

Build a comprehensive AI security testing framework for a real LLM application: a Promptfoo-based automated test suite with 50+ test cases covering all OWASP Top 10 categories, an LLM firewall proxy, a real-time security dashboard, a PII detection and redaction pipeline, and a written security report with findings and recommendations. This is a portfolio piece that demonstrates both offensive and defensive security skills.

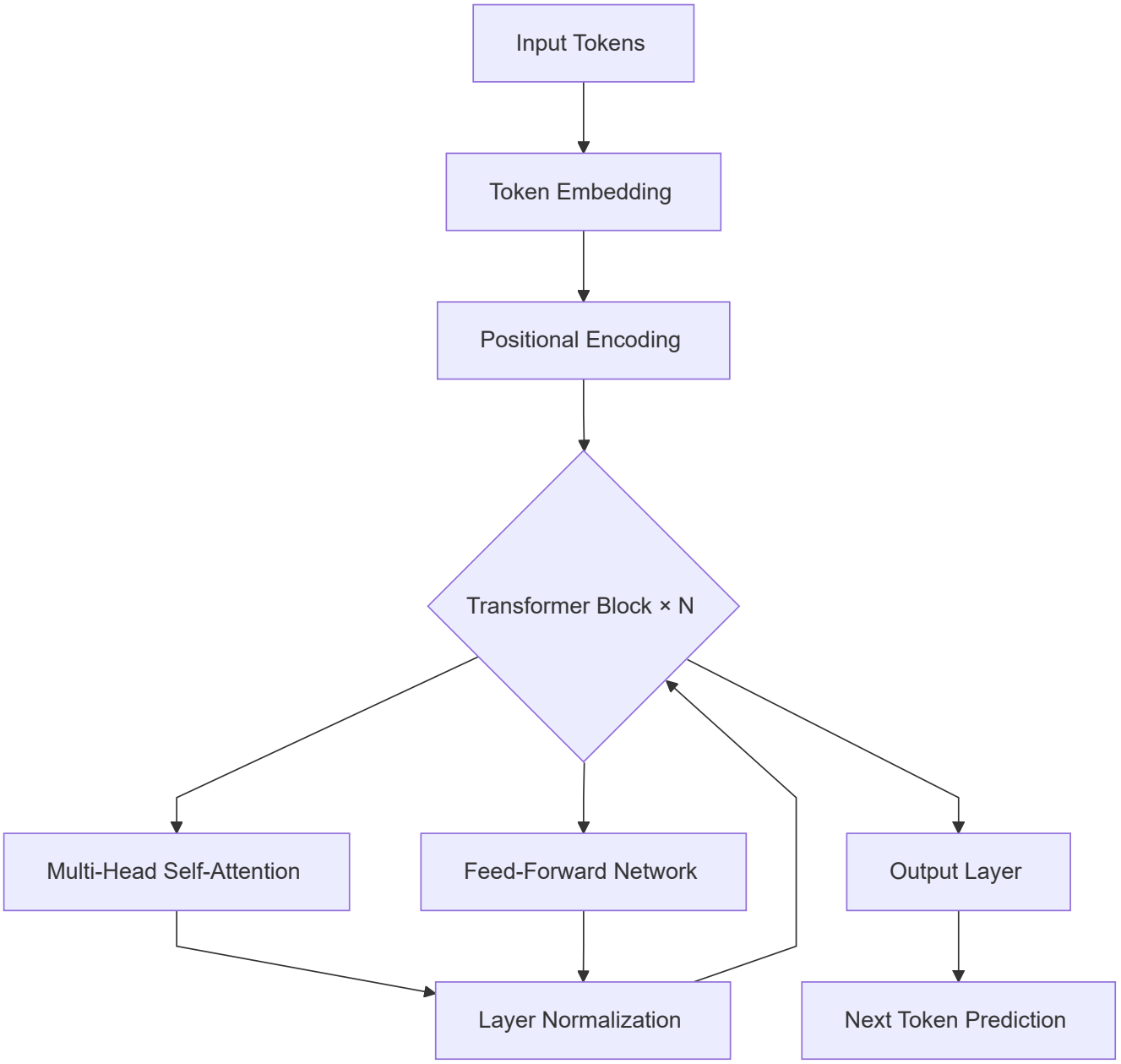
Top Open-Source Projects to Contribute To

Garak (`leondz/garak`) is the primary LLM vulnerability scanner — it needs new probes for emerging attack patterns and any security researcher's contribution is welcomed. **Guardrails AI** (`guardrails-ai/guardrails`) is the most widely used guardrails framework. **LLM Guard** (`protectai/llm-guard`) is a newer, lighter alternative. **PyRIT** (`Azure/PyRIT`) is Microsoft's Python Risk Identification Toolkit for Generative AI and is an excellent project to contribute to. **AISploit** (`hupe1980/aisploit`) is a dedicated red-teaming toolkit for AI systems.

Field 6 — LLM Architecture

What it is: You study how large language models are actually designed and built — transformers, attention mechanisms, scaling laws, and system architecture. You understand why DeepSeek, GPT, Gemini, and KIMI k2 make the design choices they do, and you can design scalable AI system infrastructure.

Core Tools: PyTorch, Hugging Face Transformers, Docker, Kubernetes, LangGraph (for system design)



Week 1 — Foundation (Days 1–7)

Day 1 — The Transformer Architecture
Read "Attention Is All You Need" (Vaswani et al., 2017) — the paper that started everything. Understand the encoder-decoder structure, why RNNs were replaced, and what the transformer's key innovation was. Then watch Andrej Karpathy's "Let's build GPT from scratch" video and implement a tiny transformer (~1M parameters) yourself in PyTorch. If you can build it, you understand it.

Day 2 — Attention Mechanisms in Depth
Self-attention lets every token look at every other token and decide which are relevant. Study: scaled dot-product attention (Q, K, V matrices), multi-head attention (run attention in parallel with different learned projections), and causal masking (in autoregressive models, tokens can only attend to past tokens). Implement all three from scratch with no PyTorch attention layers — raw matrix operations only.

Day 3 — Modern Architecture Variants
The original transformer has been heavily modified. Study: Grouped Query Attention (GQA, used in Llama 3 — fewer K/V heads for efficiency), Mixture of Experts (MoE, used in Mixtral, DeepSeek, and GPT-4 — only some "expert" layers activate per token), Flash Attention (memory-efficient attention algorithm), and RoPE (Rotary Position Embeddings, better positional encoding). Understand which models use which and why.

Day 4 — Scaling Laws
Kaplan et al.'s scaling laws paper (2020) showed that model performance scales predictably with compute, data, and parameters. Understand: compute-optimal training (Chinchilla scaling — the optimal ratio of parameters to training tokens), emergent capabilities (abilities that appear suddenly at certain scales), and inference compute (test-time compute scaling used by o1, DeepSeek R1). These laws are how labs decide what models to build.

Day 5 — Tokenization
How does text become tokens? Study BPE (Byte-Pair Encoding — the algorithm GPT uses), SentencePiece (Google's approach), and Tiktoken (OpenAI's fast implementation). Why do different languages have different token efficiency? (English is efficient; Thai and Arabic are expensive.) Implement BPE from scratch on a small corpus. Understand why tokenization is a critical design choice that affects everything downstream.

Day 6 — Inference & KV Cache
Generating text token by token is expensive. The KV Cache optimization stores the key and value tensors from past tokens so they don't need to be recomputed. Study: how the KV cache works, why it grows linearly with sequence length, techniques like PagedAttention (used in vLLM) that manage KV cache memory efficiently. This is the core engineering challenge of serving large models.

Day 7 — Sunday Presentation (Week 1)
Explain the transformer architecture using your own diagrams — no slides borrowed from the internet. Walk through your attention implementation code. The community should understand what a "head" in multi-head attention actually is.

Week 2 — Building (Days 8–14)

Day 8 — Pre-Training Data & Processing
What data makes a great LLM? Study: Common Crawl (70% of the web), The Pile, RedPajama, and FineWeb. Understand data quality filtering (deduplication, perplexity filtering, content filtering), tokenization at scale, and dataset mixing ratios (how much code vs text vs books). The quality of pre-training data determines everything about a model's capabilities.

Day 9 — Study DeepSeek Architecture
DeepSeek-V3 and R1 are the most transparent and well-documented frontier models. Read the DeepSeek-V3 technical report. Understand their MoE design (61 experts, 8 activated per token), Multi-head Latent Attention (MLA — compresses KV cache using a low-rank projection), and how they achieved GPT-4-level performance with a fraction of the training compute. This case study teaches system-level thinking.

Day 10 — Inference Systems: vLLM
vLLM is the standard serving framework for open-source LLMs. Set it up and serve Llama-3 (or Qwen) locally. Learn: PagedAttention (virtual memory for KV cache), continuous batching (process multiple requests simultaneously), tensor parallelism (split a model across multiple GPUs). Benchmark throughput (tokens/second) with different batch sizes and parallelism settings.

Day 11 — Quantization for Deployment
A Llama-3 70B model in float32 is ~280GB — too large for most hardware. Quantization reduces precision: GGUF (4-bit integer, used by llama.cpp — runs on CPU), GPTQ (4-bit, optimized for GPU inference), and AWQ (4-bit, better quality than GPTQ). Use llama.cpp to run a quantized model locally. Measure the accuracy/speed trade-off across quantization levels.

Day 12 — Designing a Scalable LLM System
Design (on paper, with diagrams) a production LLM serving infrastructure: load balancer, inference cluster (multiple vLLM instances), request queuing, autoscaling, monitoring, and fallback to cloud APIs when capacity is exceeded. Think through failure modes: what happens when one inference node goes down? How do you do a zero-downtime model update? Write this up as an architecture document.

Day 13 — Context Length Extension Techniques
Standard transformers struggle with very long contexts because attention is $O(n^2)$ in sequence length. Study: RoPE scaling (YaRN, LongRoPE — extend position embeddings beyond training length), Sliding Window Attention (Mistral's approach — attend only to a local window), and Mamba (state-space models as an alternative to transformers). Understand the trade-offs.

Day 14 — Sunday Presentation (Week 2)
Present your LLM serving architecture diagram. Show a live demo of vLLM serving a local model. Explain DeepSeek's MoE design and why it's significant.

Week 3 — Deepening (Days 15–21)

Day 15 — RLHF & Alignment Techniques
How do you make a pre-trained model helpful and safe? Study the training pipeline: SFT (Supervised Fine-Tuning on human demonstrations), RLHF (Reinforcement Learning from Human Feedback — train a reward model, then use PPO to optimize the LLM toward higher rewards), DPO (Direct Preference Optimization — simpler alternative to RLHF, used in Llama models), and GRPO (Group Relative Policy Optimization — used in DeepSeek R1 for reasoning). Read the InstructGPT paper.

Day 16 — Reasoning Models: Chain-of-Thought at Scale
OpenAI o1, DeepSeek R1, and Gemini Thinking are models trained to reason at inference time — generating long internal thinking traces before answering. Study: how long chain-of-thought improves math and coding accuracy, test-time compute scaling (more thinking = better answers, up to a point), MCTS (Monte Carlo Tree Search) as a search strategy during reasoning, and process reward models (PRMs) that give reward signals per reasoning step, not just per final answer.

Day 17 — Distributed Training Fundamentals
Training frontier models requires distributed compute. Study: data parallelism (same model, different data batches across GPUs), tensor parallelism (split model layers across GPUs), pipeline parallelism (split model by layers across GPUs), and ZeRO (Zero Redundancy Optimizer — eliminate redundant memory across GPUs). Understand why GPT-4 reportedly used ~25,000 A100s. Read the Megatron-LM and ZeRO papers.

Day 18 — Open Source Contribution Day
Contribute to **vLLM** (`vllm-project/vllm`) — the most active LLM serving project. **llama.cpp** (`ggerganov/llama.cpp`) is a foundational inference engine written in C++. **nanoGPT** (`karpathy/nanoGPT`) is educational and Karpathy himself sometimes reviews PRs. Contributing to any of these puts your name in codebases that millions of engineers use.

Day 19 — Multimodal Architecture Design
How does GPT-4V process images alongside text? Study: vision encoders (CLIP, ViT — convert images to patch embeddings), projection layers (align visual embeddings with the LLM's token space), and cross-attention (some architectures use cross-attention between vision and language features). Analyze the architecture of LLaVA (open-source multimodal LLM) by reading its code on HuggingFace.

Day 20 — Systems Optimization: Speculative Decoding
Speculative decoding uses a small fast model (draft model) to generate candidate tokens, then uses the large model to verify multiple tokens at once. This can achieve 2–3x inference speedup with zero quality loss. Implement a simple speculative decoding setup with a small Qwen and a larger Qwen model. This technique is used in production by Google and Meta.

Day 21 — Sunday Presentation (Week 3)
Present a technical deep-dive on one architecture innovation: MoE, GQA, MLA, or speculative decoding. Use diagrams you created yourself. This should feel like a mini conference talk — the DEEPSEED community should learn something they didn't know.

Week 4 — Capstone (Days 22–30)

Train a small language model from scratch (GPT-2 scale, ~125M parameters) on a domain-specific dataset of your choice using nanoGPT or a custom PyTorch implementation. Deploy it with vLLM, quantize it with GGUF, benchmark its performance, and write a technical report analyzing its capabilities and limitations. Include an architecture diagram, training loss curves, and comparison with a few-shot prompted GPT-4o-mini on your domain's benchmark tasks.

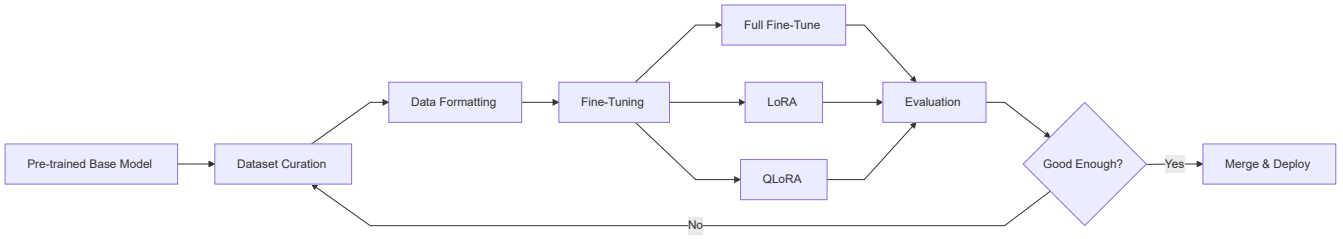
Top Open-Source Projects to Contribute To

nanoGPT (`karpathy/nanoGPT`) is the most educational codebase in the field — a minimal, readable GPT implementation perfect for learning and contributing. **vLLM** (`vllm-project/vllm`) is the most widely deployed LLM serving framework with very active development. **llama.cpp** (`ggerganov/llama.cpp`) powers offline LLM inference globally. **Transformers** (`huggingface/transformers`) by Hugging Face is the universal model hub, and any new architecture implementation is valuable. **Megatron-LM** (`NVIDIA/Megatron-LM`) is NVIDIA's framework for distributed training at scale.

Field 7 — Fine-Tuning & Model Training

What it is: You customize pre-trained language models using specialized datasets to make them excel at specific tasks. You understand parameter-efficient techniques like LoRA, can curate training data, monitor training runs, and ship models that outperform general-purpose models on your target domain.

Core Tools: Hugging Face, PyTorch, Unsloth, Axolotl, LoRA/QLoRA



Week 1 — Foundation (Days 1–7)

Day 1 — Why Fine-Tune?
Pre-trained models are generalists. Fine-tuning makes them specialists. Study the cases where fine-tuning wins: consistent output format (JSON, medical reports, legal clauses), specialized vocabulary (medical, legal, financial), task-specific behavior (classify, extract, score), and lower inference cost (a fine-tuned 7B model can outperform GPT-4 on your specific task at 10% of the cost). Also study when NOT to fine-tune: when better prompting works, when your dataset is too small (<500 examples), or when you need general reasoning.

Day 2 — Hugging Face Ecosystem
Hugging Face is the GitHub of AI models. Learn: `transformers` (load any model), `datasets` (load and process training data), `peft` (parameter-efficient fine-tuning), `trl` (training with RL), and `accelerate` (multi-GPU training made simple). Load Qwen2.5-7B and run inference. Browse the Hub and study what makes a good model card. Create your HuggingFace account and fork a dataset.

Day 3 — Dataset Curation & Formatting
Garbage in, garbage out. Learn to: scrape and clean domain-specific text, format data for instruction fine-tuning (the `{"instruction": ..., "input": ..., "output": ...}` format), deduplicate (remove near-duplicate samples using MinHash), quality filter (perplexity-based filtering, length filtering), and balance classes (equal representation of topics). A dataset of 2,000 high-quality examples beats 50,000 noisy ones.

Day 4 — LoRA: Low-Rank Adaptation
Full fine-tuning updates all model weights — expensive and risky (catastrophic forgetting). LoRA freezes the original weights and adds small trainable low-rank matrices. Learn the mathematics: if a weight matrix W is m×n, LoRA decomposes the update as ΔW = AB where A is m×r and B is r×n, with r << min(m,n). With r=16, you train <1% of parameters but get 90%+ of full fine-tune quality. Implement LoRA training using `peft`.

Day 5 — QLoRA: Fine-Tuning on Consumer Hardware
QLoRA combines quantization with LoRA to fine-tune 70B models on a single GPU. The base model is loaded in 4-bit precision (frozen), and LoRA adapters are trained in 16-bit. Use `bitsandbytes` for quantization and `peft` for LoRA. Fine-tune Llama-3.2-3B or Qwen2.5-3B on a custom dataset using Google Colab (free GPU). This is the technique that democratized fine-tuning.

Day 6 — Unsloth: Fast Fine-Tuning
Unsloth makes fine-tuning 2-5x faster and 60% more memory efficient than standard HuggingFace training through custom CUDA kernels. Set it up and replicate your QLoRA experiment from Day 5 using Unsloth. Compare: training time, GPU memory usage, and final model quality. Unsloth is now the standard tool for efficient fine-tuning.

Day 7 — Sunday Presentation (Week 1)
Explain LoRA with a visual diagram — show what the low-rank decomposition looks like geometrically. Demo your first fine-tuned model responding to prompts in your target domain. Compare its outputs to the base model on 5 test prompts.

Week 2 — Building (Days 8–14)

Day 8 — Axolotl: Production Training Configs
Axolotl is a configuration-driven fine-tuning framework that wraps HuggingFace and supports every training technique via a single YAML file. Learn the config schema: model, dataset, training args, LoRA config, evaluation. Reproduce your fine-tune using Axolotl. The benefit: reproducible, version-controlled training runs defined in config files rather than scripts.

Day 9 — Data Synthesis with LLMs
When you don't have enough real data, generate synthetic data. Use GPT-4o to generate training examples for your target task. Learn: seed data (start with 10 real examples, ask GPT-4 to generate 1,000 similar ones), self-instruct (generate instruction-output pairs from a small set of seed tasks), and constitutional AI (generate data following specific principles). Evol-Instruct (used to create WizardLM) takes existing instructions and rewrites them to be harder.

Day 10 — Instruction Tuning vs. Continued Pre-Training
Instruction tuning teaches a model to follow instructions using chat format. Continued pre-training teaches a model domain knowledge by training on raw domain text. Know when to use each: for a medical chatbot, first do continued pre-training on medical literature to give it knowledge, then instruction-tune it to follow doctor commands helpfully. Implement both on a small model.

Day 11 — DPO: Direct Preference Optimization
DPO trains a model to prefer "chosen" outputs over "rejected" outputs using paired comparison data. Collect preference data: for each prompt, write two responses — a better one and a worse one. Train with `trl`'s DPO trainer. DPO is simpler than RLHF and produces better-aligned models. Fine-tune a model with DPO to be more helpful and less verbose.

Day 12 — Evaluation-Driven Fine-Tuning
Define your evaluation benchmark BEFORE training. Create 50 test examples with expected answers. After each training epoch, run evaluation and plot the metrics. Use: exact match for structured outputs, ROUGE/BLEU for text generation, custom LLM-as-judge scoring for open-ended tasks. Training without evaluation is flying blind.

Day 13 — Merging & Quantizing Your Fine-Tuned Model
After training LoRA adapters, merge them into the base model for easier deployment. Use `peft`'s `merge_and_unload()`. Then quantize to GGUF format using `llama.cpp`'s `convert.py`. Upload the merged and quantized model to HuggingFace Hub. Your model is now publicly available and anyone in the world can run it — your first public AI contribution.

Day 14 — Sunday Presentation (Week 2)
Demo your DPO-trained model vs. the base model on preference-sensitive prompts. Show the evaluation curves from training. Walk through your HuggingFace model card and explain the data card alongside it.

Week 3 — Deepening (Days 15–21)

Day 15 — Multi-Task Fine-Tuning
Train one model to do multiple tasks well simultaneously. The challenge is catastrophic forgetting: fine-tuning on task B can make the model forget task A. Study: multi-task learning (mix datasets from all tasks), LoRA merging (train separate LoRA for each task, merge with TIES or DARE), and mixture-of-LoRA (route to different LoRA adapters per task). Build a model that can summarize, classify, and extract entities.

Day 16 — Training Instability & Debugging
Fine-tuning goes wrong in many ways: loss spikes (learning rate too high), mode collapse (model produces repetitive outputs), gradient explosion (clip gradients), overfitting (too many epochs on small dataset). Learn to diagnose each failure mode from the loss curves and training logs. Use Weights & Biases (wandb) to log and visualize every training run.

Day 17 — GRPO: Training Reasoning Models
GRPO (Group Relative Policy Optimization) is the technique used to train DeepSeek R1 to reason. For each prompt, generate a group of N responses, score them using a reward model or rule-based verifier (e.g., "is the math correct?"), and use the relative scores within the group to compute policy gradients. Implement GRPO training using `trl`'s GRPO trainer on a math reasoning dataset.

Day 18 — Open Source Contribution Day
Contribute to **Unsloth** (`unslothai/unsloth`) — they always need help with new model support, documentation, and benchmarks. **Axolotl** (`OpenAccess-AI-Collective/axolotl`) needs config examples and documentation. **TRL** (`huggingface/trl`) by Hugging Face is the training library with the widest reach. Submitting a dataset to the HuggingFace Hub for a domain with sparse training data is also a genuine contribution.

Day 19 — Serving Your Fine-Tuned Model
Serving a custom model requires different infrastructure than calling an API. Learn: `ollama` (simplest local serving), vLLM (production serving with continuous batching), and Hugging Face Inference Endpoints (managed cloud serving). Build a FastAPI wrapper around your fine-tuned model with the same interface as the OpenAI API — so your app can switch between your model and GPT-4 with one line change.

Day 20 — Domain Adaptation at Scale
Study how frontier fine-tunes are built: CodeLlama (fine-tuned on code), BioMedLM (fine-tuned on biomedical text), Lawyer-Llama (fine-tuned on legal text). Pick one open model and read its technical report. What data did they use? What training technique? What were the key challenges? Write a 2-page analysis. Understanding how others solved the problem teaches you how to solve yours.

Day 21 — Sunday Presentation (Week 3)
Present your GRPO-trained reasoning model. Show it solving math problems step by step. Compare its accuracy against the base model on a benchmark. Explain how GRPO differs from RLHF and why it's more stable.

Week 4 — Capstone (Days 22–30)

Fine-tune a model for a real-world domain you care about: medical Q&A, legal document analysis, code generation in a niche language, or customer support for a specific product vertical. The capstone must include: a curated dataset (minimum 1,000 examples) with a data card, QLoRA + DPO fine-tuning with evaluation-driven iteration, a quantized and merged model on HuggingFace Hub, a benchmark comparing your model against

the base model and GPT-4o-mini, and a deployed inference endpoint.

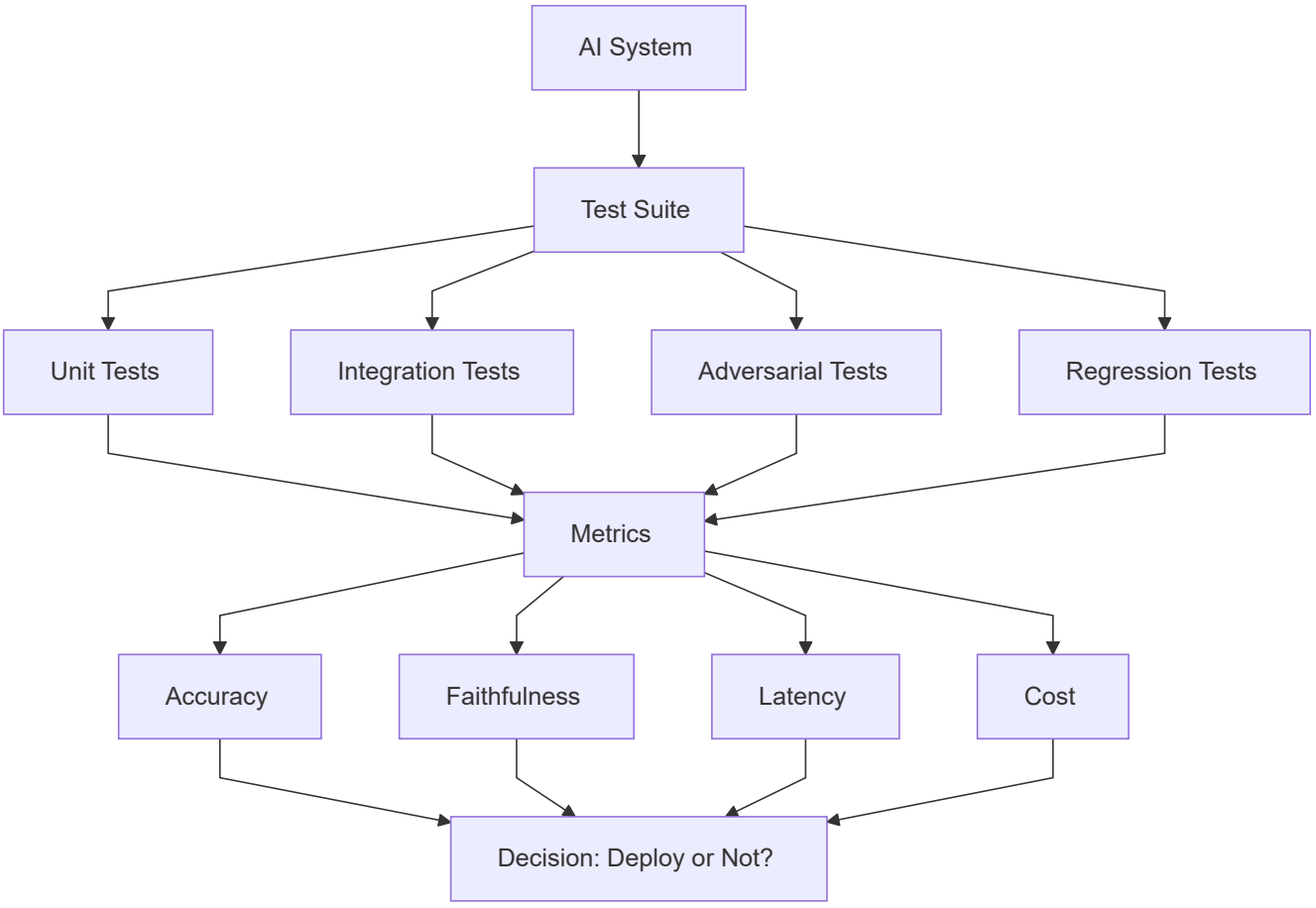
Top Open-Source Projects to Contribute To

Unsloth (`unslothai/unsloth`) is the fastest-growing fine-tuning library and the team actively engages with contributors. **Axolotl** (`OpenAccess-AI-Collective/axolotl`) is the most configurable training framework and welcomes new example configs and documentation. **TRL** (`huggingface/trl`) is the standard for RLHF/DPO/GRPO training. **LLaMA-Factory** (`hiyouga/LLaMA-Factory`) supports more models and techniques than almost any other framework and has an active Chinese and international community. **Open-Instruct** (`allenai/open-instruct`) from Allen AI is the research-grade training framework used to produce Tulu models.

Field 8 — AI Evaluation & Testing

What it is: You measure whether AI systems actually work. You design test suites, build evaluation pipelines, measure accuracy and reliability, detect regressions, and give teams the evidence they need to make confident deployment decisions. Evaluation is what separates AI engineering from AI guessing.

Core Tools: LangSmith, DeepEval, Ragas, Promptfoo



Week 1 — Foundation (Days 1–7)

Day 1 — The Evaluation Problem
Why is evaluating LLMs hard? They produce open-ended text, "correct" is subjective, they are non-deterministic, and they can produce plausible-sounding wrong answers. Study the three types of evaluation: human evaluation (gold standard, slow, expensive), automatic metrics (ROUGE, BLEU, BERTScore — fast but imperfect), and LLM-as-judge (use a strong LLM to score outputs — the current best practice). Understand the limitations of each and when to use which.

Day 2 — Building a Test Dataset
Good evaluation starts with good test data. Learn: golden dataset (expert-curated question-answer pairs), adversarial examples (edge cases designed to break the system), boundary cases (ambiguous inputs where behavior should be defined), and regression cases (previously failing inputs that should now pass). Build a 50-example golden dataset for a specific task — this is harder and more valuable than it sounds.

Day 3 — LLM-as-Judge: Theory & Implementation
The most powerful evaluation technique: use a strong LLM (GPT-4o or Claude) as an automated evaluator. Learn: direct scoring (rate on a 1-5 scale with rubric), comparative evaluation (given two answers, which is better?), and reference-based evaluation (compare against a golden answer). The key challenge is prompt design — the evaluation prompt must be explicit, consistent, and calibrated against human judgments. Implement all three and measure inter-rater agreement with human labels.

Day 4 — DeepEval: Automated LLM Testing
DeepEval is the pytest of LLM testing. Learn its metric library: `GEval` (custom LLM-based metric), `AnswerRelevancy`, `Faithfulness`, `Hallucination`, `ContextualPrecision`. Write a test suite that runs automatically: define test cases, run evals, set thresholds, assert pass/fail. Integrate with `pytest` so your CI/CD pipeline runs evaluations on every code change.

Day 5 — Ragas: RAG-Specific Evaluation
Ragas provides metrics purpose-built for RAG systems: answer relevancy (does the answer address the question?), faithfulness (is the answer grounded in the retrieved context?), context precision (is the retrieved context relevant?), and context recall (was all necessary context retrieved?). Set up Ragas on a RAG system and measure all four metrics. These four numbers tell you exactly where your RAG is failing.

Day 6 — LangSmith: Observability for LLM Apps
LangSmith traces every LLM call in your application: inputs, outputs, latency, cost, and model used. Set it up with your LangChain or direct API calls. Learn to: add custom evaluation criteria to traces, compare prompt versions A/B in LangSmith, tag and filter traces, and build dashboards. Production evaluation requires observability, not just offline testing.

Day 7 — Sunday Presentation (Week 1)
Demo your DeepEval test suite running automatically. Show LangSmith traces for a real application. Explain why LLM-as-judge is powerful but also where it can mislead (biases toward longer answers, sycophancy toward its own outputs).

Week 2 — Building (Days 8–14)

Day 8 — Promptfoo for Model Comparison
Promptfoo lets you test the same prompts against multiple models and compare results. Build a head-to-head comparison: same 50 test cases, run against GPT-4o, Claude Sonnet, and Gemini Pro. Score each automatically with an LLM judge. Produce a comparison report. This is how teams decide which model to use in production — with data, not gut feeling.

Day 9 — Hallucination Detection
Hallucination is when a model generates confident-sounding incorrect information. Build a hallucination detector: given a claim and a source document, does the claim contradict or go beyond the source? Use `FEVER` dataset for training. Study: NLI (Natural Language Inference) for factual consistency checking, SelfCheckGPT (generate the same output multiple times — if the model disagrees with itself, it's likely hallucinating), and RAGTruth benchmark. This is one of the most important evaluation skills.

Day 10 — Regression Testing & CI/CD Integration
When you update a model, prompt, or retrieval system, you need to know if it got better or worse. Build a regression test suite in GitHub Actions: on every PR, run 50 evaluation cases, compare scores against the baseline, and fail the PR if there's a significant regression. This brings software engineering discipline to AI development.

Day 11 — Human Evaluation Pipelines
Automated metrics aren't enough for user-facing products. Build a human evaluation pipeline: a Streamlit app where evaluators rate model outputs on specific dimensions, inter-annotator agreement tracking (Cohen's Kappa), and an annotation guideline document. Study how RLHF data collection works at scale. Even a team of 5 evaluators needs a disciplined process.

Day 12 — Benchmarking with Standard Datasets
Learn the standard AI benchmarks and what they measure: MMLU (general knowledge, 57 subjects), HumanEval (code generation), GSM8K (grade school math reasoning), MT-Bench (multi-turn conversation quality), HellaSwag (commonsense reasoning), and TruthfulQA (truthfulness). Evaluate an open-source model on at least two of these benchmarks using `lm-evaluation-harness`.

Day 13 — Evaluation for Agents
Agent evaluation is harder than LLM evaluation because you need to evaluate multi-step processes. Study: task completion rate (did the agent finish the task?), step accuracy (were individual tool calls correct?), efficiency (how many steps did it take?), and robustness (does the agent fail gracefully on edge cases?). Build an agent evaluation harness using WebArena or a custom environment with known ground truth.

Day 14 — Sunday Presentation (Week 2)
Present your model comparison report from Promptfoo. Show your CI/CD regression testing pipeline catching a real regression. Explain hallucination detection with concrete examples.

Week 3 — Deepening (Days 15–21)

Day 15 — Calibration & Uncertainty
A well-calibrated model that says "I'm 80% confident" should be right 80% of the time. Study: Expected Calibration Error (ECE), temperature scaling (adjusting model confidence post-hoc), conformal prediction (generate prediction sets with guaranteed coverage). Build a calibration plot for an LLM classification task and measure how well-calibrated different models are.

Day 16 — Red Teaming as Evaluation
Evaluation should include adversarial testing — systematically trying to break the system. Build a red team evaluation suite: prompt injections, jailbreak attempts, out-of-distribution inputs, edge cases in the distribution, and deliberately ambiguous queries. Measure your system's robustness score. Good evaluation includes trying to fail, not just trying to succeed.

Day 17 — Evaluation Dataset Generation
When you can't find a suitable benchmark dataset, build one. Learn: LLM-assisted annotation (use GPT-4 to generate candidates, humans to verify), active learning (prioritize examples where the model is most uncertain), and minimum viable dataset size calculation (use statistical power analysis). Build a synthetic evaluation dataset for a domain-specific task.

Day 18 — Open Source Contribution Day
DeepEval (`confident-ai/deepeval`) actively needs new metric implementations and model integrations. **Ragas** (`explodinggradients/ragas`) needs metric improvements and better documentation. **lm-evaluation-harness** (`EleutherAI/lm-evaluation-harness`) is the standard academic benchmarking tool and welcomes new task implementations. **PromptBench** (`microsoft/promptbench`) from Microsoft needs more adversarial prompt templates.

Day 19 — Evaluation at Scale
Running 50 evaluations manually is fine for development. Running 5,000 evaluations nightly requires infrastructure. Build a scalable eval pipeline: batch LLM judge calls with async processing, store results in SQLite or PostgreSQL, build aggregation queries, and generate weekly evaluation reports automatically. This is what a "model quality team" at a tech company does.

Day 20 — Evaluation Pitfalls & Goodhart's Law
"When a measure becomes a target, it ceases to be a good measure." Study common evaluation failures: benchmark contamination (model was trained on test data), metric gaming (model learns to maximize metric without improving quality), annotation artifacts (test sets have spurious patterns that models exploit), and distribution shift (test set doesn't reflect real user behavior). Write a blog post (for your GitHub) on evaluation pitfalls you've encountered.

Day 21 — Sunday Presentation (Week 3)
Present your automated nightly eval pipeline. Show a calibration analysis of two different models. This session should feel academic — the community should leave understanding evaluation at a deeper level than "run some tests."

Week 4 — Capstone (Days 22–30)

Build a comprehensive evaluation framework for a real AI system (a RAG pipeline, a chatbot, or an agent). The framework must include: a 100-example annotated golden dataset, automated metrics (DeepEval + Ragas + custom LLM judge), a CI/CD integration that runs on every PR, a human evaluation Streamlit app, a regression tracking dashboard (LangSmith or custom), and a written evaluation report analyzing the system's strengths and weaknesses.

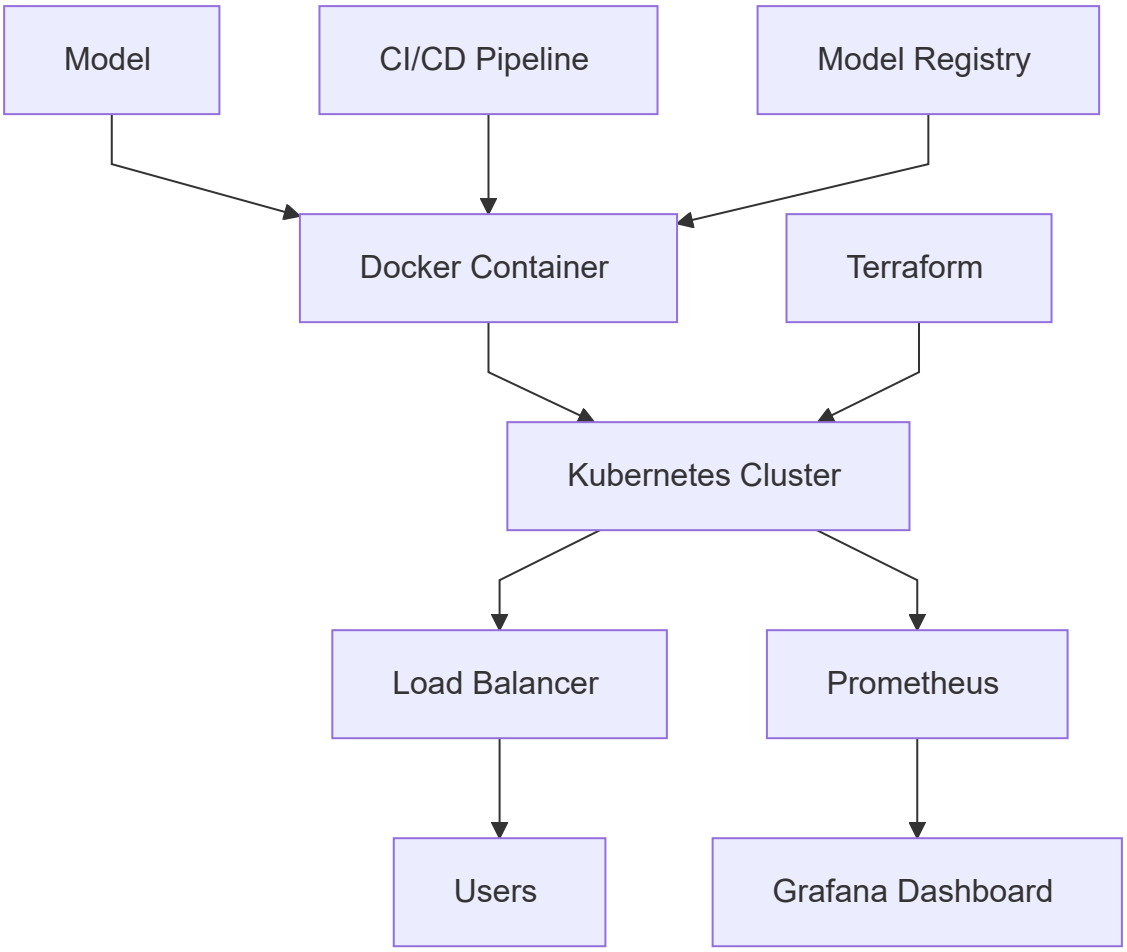
Top Open-Source Projects to Contribute To

DeepEval (`confident-ai/deepeval`) is the fastest-growing LLM testing framework. **lm-evaluation-harness** (`EleutherAI/lm-evaluation-harness`) is the standard academic benchmarking tool used by virtually every AI lab. **Ragas** (`explodinggradients/ragas`) is the go-to RAG evaluation library. **Evals** (`openai/evals`) is OpenAI's eval framework — contributing here is visible to the broader research community. **HELM** (`stanford-crfm/helm`) from Stanford is a rigorous holistic evaluation framework.

Field 9 — AI Infrastructure (AI Ops)

What it is: You deploy, scale, monitor, and maintain AI systems in production. You bridge the gap between AI research and reliable products by building the infrastructure that keeps models running, fast, cheap, and observable.

Core Tools: Docker, Kubernetes, AWS/Azure/GCP, Terraform, Prometheus, Grafana



Week 1 — Foundation (Days 1–7)

Day 1 — Docker for AI Systems
Docker containers are how AI services are packaged and deployed. Build a Dockerfile for an LLM API: base image (Python 3.11-slim), install dependencies, copy your FastAPI app, expose port 8000, set environment variables for API keys. Build the image, run it locally, and verify it works identically to your development environment. Understand: images vs containers, layers, caching, and multi-stage builds for smaller images.

Day 2 — Docker Compose for Local AI Stacks
Production AI stacks have multiple services. Build a `docker-compose.yml` that runs: your FastAPI LLM service, Qdrant vector database, Redis for caching, and PostgreSQL for application data. Define health checks, volume mounts (so data persists across restarts), and a shared network. This is the local development equivalent of a production Kubernetes setup.

Day 3 — Cloud Fundamentals: AWS
Learn the AWS services used most in AI systems: EC2 (GPU instances — g4dn.xlarge for inference, p3 for training), ECS/Fargate (container orchestration without managing nodes), S3 (store models, datasets, logs), RDS (managed PostgreSQL), ElastiCache (managed Redis), and IAM (identity and access management — never use root credentials). Set up an AWS Free Tier account and deploy your Docker container to ECS Fargate.

Day 4 — Kubernetes Basics
Kubernetes is the standard for orchestrating containerized AI systems at scale. Learn: Pods (running containers), Deployments (manage replicas), Services (expose deployments to traffic), ConfigMaps and Secrets (configuration without hardcoding), and HorizontalPodAutoscaler (auto-scale based on CPU/memory). Deploy your LLM API to a local Kubernetes cluster using `minikube` or `kind`.

Day 5 — Terraform: Infrastructure as Code
Manually clicking through AWS console is not engineering. Terraform defines your infrastructure in code: write `.tf` files that specify EC2 instances, security groups, RDS databases, and S3 buckets. `terraform apply` creates everything. `terraform destroy` removes everything. Build Terraform configs for a basic AI service stack. Infrastructure as Code means your infrastructure is version-controlled, reproducible, and auditable.

Day 6 — Monitoring: Prometheus & Grafana
You cannot operate what you cannot see. Prometheus scrapes metrics from your services (request rate, latency, error rate, GPU utilization, model inference time). Grafana visualizes them in dashboards. Set up both with Docker Compose. Add `prometheus-fastapi-instrumentator` to your FastAPI app to expose metrics automatically. Build a dashboard with the golden signals: latency, traffic, errors, and saturation.

Day 7 — Sunday Presentation (Week 1)
Demo your full local AI stack running with Docker Compose. Show the Grafana dashboard responding to live traffic. Walk through your Terraform config and explain what each resource does.

Week 2 — Building (Days 8–14)

Day 8 — GPU Infrastructure for AI
CPUs are for small models. GPUs are for everything else. Study: NVIDIA GPU families (T4 for inference, A100/H100 for training), CUDA and cuDNN (the software stack), GPU memory management (VRAM is the bottleneck, not compute), and multi-GPU setups (NVLink for inter-GPU communication). Set up a GPU-enabled Docker container with CUDA support and benchmark inference speed with and without GPU.

Day 9 — Model Serving with vLLM on Kubernetes
Deploy vLLM as a Kubernetes Deployment serving an open-source LLM. Configure: resource requests (GPU memory, CPU), liveness and readiness probes (Kubernetes needs to know when your model is loaded and ready), horizontal pod autoscaling (scale out when queue depth exceeds threshold), and persistent volume claims (cache model weights between restarts). This is production LLM infrastructure.

Day 10 — CI/CD for AI Systems
AI systems need CI/CD just like software systems. Build a GitHub Actions pipeline: on PR, run linting and unit tests → run automated LLM evaluations → build Docker image → push to ECR (Elastic Container Registry) → deploy to staging environment → run smoke tests → require manual approval → deploy to production. AI deployment without CI/CD is ops theater.

Day 11 — Model Registry & Versioning
You need to track model versions: which model is in production, what commit trained it, what dataset it used, and what evaluation scores it achieved. Set up MLflow as a model registry: log training runs with parameters and metrics, register model versions, and tag the production version. Alternatively, use Hugging Face private spaces as a model registry. Build a deployment pipeline that always pulls the registered production model.

Day 12 — Observability: Distributed Tracing
In a microservices AI stack, a single user request might touch the API gateway, the LLM service, the vector database, and the cache. Distributed tracing follows that request across all services. Set up OpenTelemetry with Jaeger: instrument your FastAPI service, add trace context propagation to outbound calls, and visualize full request traces. Debugging production issues without tracing is guesswork.

Day 13 — Cost Optimization in the Cloud
AI infrastructure is expensive. Learn: Spot Instances (up to 90% discount, but can be interrupted — use for training), Reserved Instances (commit 1–3 years for 40–60% discount — use for stable production inference), right-sizing (don't pay for a p3.16xlarge if a g4dn.xlarge is sufficient), and auto-scaling (scale to zero during off-hours). Build a cost analysis for your infrastructure and identify the top three cost reduction opportunities.

Day 14 — Sunday Presentation (Week 2)
Demo your CI/CD pipeline running end-to-end — show a PR triggering evaluations, building an image, and deploying. Walk through the cost analysis and where you'd cut costs at scale.

Week 3 — Deepening (Days 15–21)

Day 15 — Kubernetes for Production AI: Advanced
Learn advanced Kubernetes for AI: KEDA (event-driven autoscaling based on queue depth — perfect for LLM inference), node affinity (schedule GPU pods on GPU nodes only), pod disruption budgets (ensure availability during node maintenance), and cluster autoscaler (add/remove nodes based on pending pods). Build a production-grade Kubernetes manifest for your LLM service.

Day 16 — Multi-Region & High Availability
Production AI systems must be available 99.9%+ of the time. Design a multi-region deployment: primary region (us-east-1), failover region (eu-west-1), global load balancer (Route 53 with health checks), data replication for the vector store, and a runbook for regional failover. Write the architecture document. You won't implement this fully, but designing it teaches you enterprise AI Ops.

Day 17 — Logging, Alerting & Incident Response
Build a complete observability stack: structured JSON logging from all services → ELK stack (Elasticsearch, Logstash, Kibana) or CloudWatch Logs for aggregation → Grafana alerts that fire PagerDuty when p99 latency > 2s or error rate > 1% → on-call runbook with step-by-step debugging procedures. Write a post-mortem template. The difference between a junior and senior AI Ops engineer is whether they have runbooks.

Day 18 — Open Source Contribution Day
Contribute to **BentoML** (`bentoml/bentoml`) — the most polished ML serving framework with active development. **Ray Serve** (`ray-project/ray`) is the distributed ML framework from Anyscale, with great architecture for learning. **KServe** (`kserve/kserve`) is the Kubernetes-native model serving standard. **MLflow** (`mlflow/mlflow`) is the universal model tracking tool used by almost every ML team.

Day 19 — AI on the Edge
Not all AI inference happens in the cloud. Edge deployment (on devices, in browsers, on-premise) is growing rapidly. Study: ONNX (universal model format for cross-platform deployment), llama.cpp for CPU inference, WebLLM (LLMs in the browser via WebGPU), and TensorRT for optimized NVIDIA inference. Deploy a quantized model to run on a Raspberry Pi or in a browser. This opens doors to healthcare, manufacturing, and offline use cases.

Day 20 — Platform Engineering for AI Teams
Senior AI Ops engineers build internal platforms that make other teams productive. Design an "AI Platform" for your DEEPSEED community: a portal where community members can deploy their fine-tuned models, a shared evaluation service, a model registry, and a cost dashboard. This is exactly what ML Platform teams at Airbnb, Uber, and LinkedIn have built.

Day 21 — Sunday Presentation (Week 3)
Present your production Kubernetes configuration with all the advanced features. Show a real alert firing and walk through the debugging runbook. The community should see AI Ops as a serious engineering discipline.

Week 4 — Capstone (Days 22–30)

Deploy a complete production AI system on AWS or GCP: containerized LLM API + vector database + Redis cache + PostgreSQL, orchestrated with Kubernetes, deployed with Terraform, monitored with Prometheus + Grafana, traced with OpenTelemetry, deployed via GitHub Actions CI/CD, with a model registry in MLflow, and a cost dashboard. Write a runbook for common operational scenarios (high latency, service crash, model quality regression). This capstone is a reference architecture.

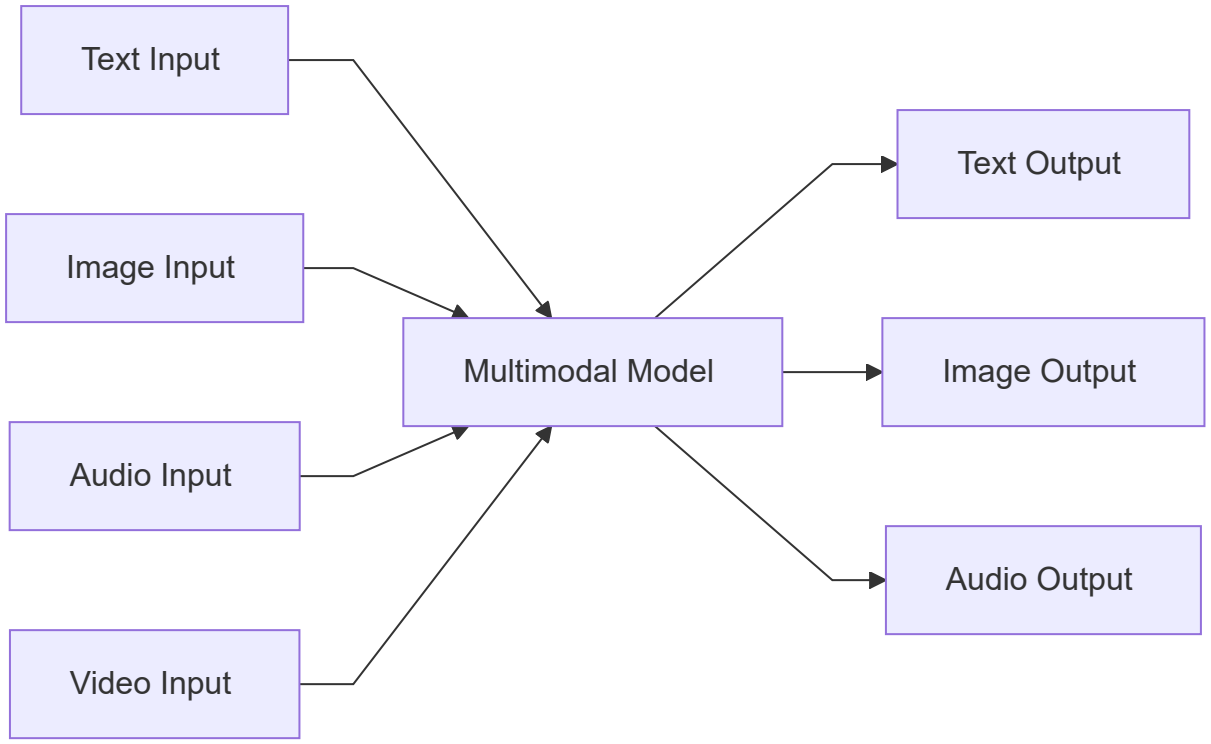
Top Open-Source Projects to Contribute To

vLLM (`vllm-project/vllm`) is the most important LLM serving project — contributing here has massive impact. **BentoML** (`bentoml/bentoml`) is the most developer-friendly ML serving framework. **Ray** (`ray-project/ray`) is the distributed computing backbone for many AI systems. **MLflow** (`mlflow/mlflow`) is universal ML tracking. **Kubeflow** (`kubeflow/kubeflow`) is the Kubernetes-native ML platform used in enterprise. **Argo Workflows** (`argoproj/argo-workflows`) is increasingly used for AI pipeline orchestration.

Field 10 — Multimodal AI

What it is: You build AI systems that understand and generate across modalities — text, images, audio, and video. Multimodal AI is where the most exciting frontier research is happening and where products like GPT-4o, Gemini, and Whisper are pushing the boundaries.

Core Tools: GPT-4o (vision), Gemini (multimodal), Whisper (audio), CLIP, PyTorch



Week 1 — Foundation (Days 1–7)

Day 1 — The Multimodal Landscape
Study the major multimodal models and what they handle: GPT-4o (text + vision input, text output), Gemini 1.5 Pro (text + vision + audio + video input, long context), Claude Sonnet (text + vision input), DALL-E 3 (text to image), Stable Diffusion (text/image to image), Whisper (audio to text), MusicGen (text to audio), and Sora/Runway (text/image to video). Map these on a grid of input modalities vs output modalities. This mental model orients everything else.

Day 2 — Vision: Image Understanding with GPT-4o
GPT-4o can analyze images in detail. Learn the API: send images as base64-encoded strings or URLs in the message content array. Build applications: image captioning (describe what's in an image), OCR (extract text from screenshots), chart analysis (read data from graphs), medical image triage (describe what's unusual in an X-ray). Study the prompt patterns that work best for each vision task.

Day 3 — CLIP: Understanding Vision-Language Alignment
CLIP (Contrastive Language-Image Pre-Training from OpenAI) is the foundation of modern vision-language AI. It embeds images and text into the same vector space so similar images and texts are close together. Build: zero-shot image classifier (ask "is this a cat or dog" by comparing CLIP similarities), image search using text queries, and visual question answering. Understanding CLIP is understanding how vision meets language.

Day 4 — Speech to Text with Whisper
OpenAI's Whisper is the best open-source speech recognition model. Run it locally using `openai-whisper`. Transcribe audio files in English, then test multilingual transcription (Whisper supports 99 languages). Build a real-time transcription service that listens to a microphone and transcribes continuously using `pyaudio` + Whisper. Study Whisper's architecture: it is a seq2seq transformer trained on 680k hours of audio.

Day 5 — Text-to-Image: Stable Diffusion
Stable Diffusion is the most important open-source image generation model. Run it locally with `diffusers`. Learn: text-to-image (generate from a prompt), image-to-image (modify an existing image with a prompt), inpainting (edit only a masked region), and ControlNet (control composition with edge maps, poses, or depth maps). Study the underlying diffusion process: images start as noise and are iteratively denoised guided by a text embedding.

Day 6 — Multimodal RAG
RAG works for images too. Build a system that indexes product images by generating text descriptions with GPT-4o, stores them in a vector database, and retrieves relevant images when a user searches with text or another image. Add PDF indexing that handles both the text and any figures in the PDF. This is the foundation of multimodal enterprise knowledge bases.

Day 7 — Sunday Presentation (Week 1)
Demo your multimodal applications: live transcription, image analysis, image generation. Explain CLIP's role in vision-language alignment using a visual demo of similar images and texts clustering together.

Week 2 — Building (Days 8–14)

Day 8 — Video Understanding
Gemini 1.5 Pro can process video natively (up to 1 hour of video). Build: a video summarization tool (upload a lecture, get structured notes), a sports analysis agent (describe what happens in each play), and a video QA system (ask questions about a video). Also study OpenAI's frame-by-frame approach: extract frames with OpenCV, embed each with CLIP, and use them for video search.

Day 9 — Text-to-Speech & Voice Cloning
Build AI voice applications with: OpenAI TTS API (highest quality), Coqui TTS (open-source, custom voices), and ElevenLabs API (voice cloning). Create a podcast generator: take a blog post, convert it to a dialogue between two speakers, generate audio for each speaker, and merge the tracks. Voice AI is one of the most practically impactful areas in multimodal.

Day 10 — LLaVA: Open-Source Multimodal LLMs
LLaVA (Large Language and Vision Assistant) is the leading open-source multimodal model. It combines a CLIP vision encoder with Llama via a projection MLP. Run LLaVA locally with Ollama: `ollama pull llava`. Build a visual assistant that can describe images, read charts, and answer questions about photos — all running locally without any API calls. Study the LLaVA architecture paper to understand how vision tokens are projected into the LLM's space.

Day 11 — Audio Classification & Understanding
Beyond speech-to-text, AI can understand audio: classify music genres, detect environmental sounds, identify speakers, and detect emotions in speech. Use `librosa` for audio feature extraction (MFCCs, spectrograms), `transformers` for pre-trained audio models (Wav2Vec2, Audio Spectrogram Transformer), and build a sound event detection system. This opens doors to accessibility, manufacturing, and healthcare applications.

Day 12 — Multimodal Applications in Practice
Build two real applications: (1) a visual QA chatbot for e-commerce where users upload product photos and ask questions about them, and (2) a meeting assistant that records audio (Whisper transcription), identifies action items (LLM extraction), and generates a summary with timestamps. These are production patterns used at real companies.

Day 13 — Evaluation of Multimodal Systems
How do you measure multimodal quality? Study: VQA accuracy (Visual Question Answering benchmark), MME (multimodal evaluation benchmark), and FID score (quality of generated images). Build an evaluation suite for your visual QA chatbot using VQA v2 dataset. Image generation quality is harder to measure automatically — study human preference studies for generative models.

Day 14 — Sunday Presentation (Week 2)
Demo your meeting assistant — record a short meeting live and show the AI producing the transcript and action items. Show LLaVA running locally with no API calls. The community should see that state-of-the-art multimodal AI can run on consumer hardware.

Week 3 — Deepening (Days 15–21)

Day 15 — Multimodal Fine-Tuning
Fine-tune a multimodal model for a specialized vision task. Use LLaVA-1.5 or Qwen2-VL with LoRA adapters. Create a dataset of image-instruction-response triples for your domain (medical images, satellite imagery, product photos). Train with Unsloth's multimodal support or the LLaVA fine-tuning scripts. Evaluate on held-out examples with LLM-as-judge scoring.

Day 16 — Diffusion Model Architecture
Study the architecture of diffusion models in depth: the U-Net backbone (encodes image with downsampling, decodes with upsampling, skip connections), cross-attention conditioning (text embeddings guide each denoising step via cross-attention), classifier-free guidance (blend conditional and unconditional denoising for quality vs diversity trade-off), and SDXL improvements over SD 1.5. Read the DDPM paper and the Stable Diffusion technical report.

Day 17 — Building a Multimodal Agent
Combine vision, language, and action in a single agent. Build an agent that: takes a screenshot of a webpage, uses GPT-4o to understand its content, generates Python code to interact with it using `playwright`, executes the code, and observes the new screenshot to decide the next action. This is essentially a computer-use agent — the foundation of products like Claude Computer Use.

Day 18 — Open Source Contribution Day
LLaVA (`haotian-liu/LLaVA`) is the reference multimodal model implementation. **Whisper** (`openai/whisper`) by OpenAI needs Python packaging improvements and additional language support contributions. **Diffusers** (`huggingface/diffusers`) is the HuggingFace library for diffusion models — one of the most active and welcoming repositories. **CLIP** (`openai/CLIP`) needs applications and fine-tuning examples. **CogVLM** (`THUDM/CogVLM`) is a strong alternative vision-language model from Tsinghua University.

Day 19 — Real-Time Multimodal Applications
Real-time multimodal AI is the hardest engineering challenge: streaming audio transcription while generating a live response, analyzing video frames as they arrive, and voice-to-voice conversation with low latency. Study the OpenAI Realtime API architecture. Build a voice assistant that transcribes speech in real time, generates a response, and speaks it back — with under 1 second total latency.

Day 20 — Multimodal Safety & Ethics
Multimodal models introduce new safety challenges: generating harmful or CSAM images, deepfakes, and voice cloning for fraud. Study: content moderation APIs (OpenAI Moderation, Google Safe Search, Microsoft Content Safety), watermarking for AI-generated images (C2PA standard), and synthetic media detection. Build a content safety pipeline for a user-generated image submission system.

Day 21 — Sunday Presentation (Week 3)
Present your multimodal agent — show it navigating a webpage using vision and action. Demo real-time voice conversation. The community should see multimodal AI as a complete system, not just individual APIs.

Week 4 — Capstone (Days 22–30)

Build a multimodal product that demonstrates mastery across at least three modalities: a visual meeting assistant (audio → transcript → image-enhanced notes → voice summary), a multimodal search engine (search across text, images, and audio from a document collection), or a multimodal content creation pipeline (user uploads images, AI generates product descriptions, and narrates them as audio). Deploy it as a full-stack application.

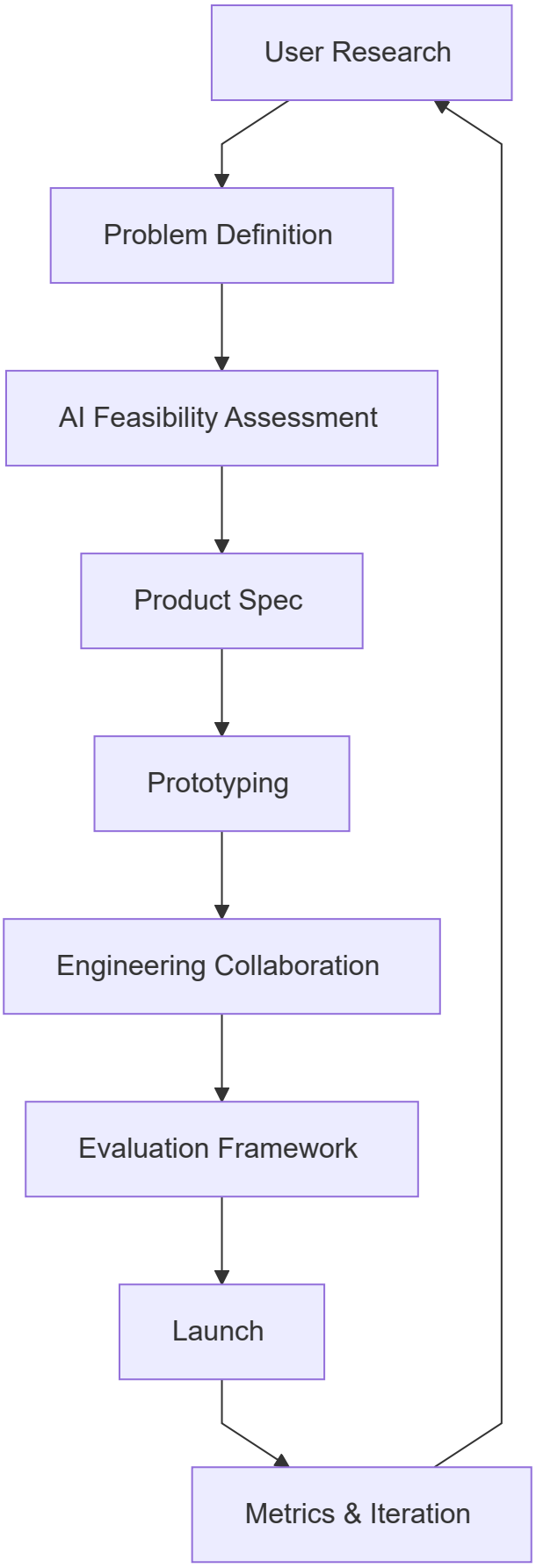
Top Open-Source Projects to Contribute To

Diffusers (`huggingface/diffusers`) is the most active multimodal open-source project and has thorough contribution documentation. **Whisper** (`openai/whisper`) powers much of the world's speech recognition. **LLaVA** (`haotian-liu/LLaVA`) is the reference vision-language implementation. **CogVLM2** (`THUDM/CogVLM2`) is a strong research model. **ComfyUI** (`comfyanonymous/ComfyUI`) is the most popular Stable Diffusion UI with a node-based pipeline — contributing UI nodes or model integrations has enormous user reach.

Field 11 — AI Product Management

What it is: You guide AI products from idea to launch to iteration. You understand enough about AI to speak fluently with engineers and researchers, enough about business to connect AI capabilities to real value, and enough about users to ensure AI actually helps people. AI PMs are among the most in-demand roles in the industry.

Core Tools: Jira, Notion, Figma, Mixpanel, AI Prototyping Platforms


User Research
Problem Definition
AI Feasibility Assessment
Product Spec
Prototyping
Engineering Collaboration
Evaluation Framework
Launch
Metrics & Iteration

Week 1 — Foundation (Days 1–7)

Day 1 — The AI PM Role

Read "Inspired" by Marty Cagan (product management foundation), then study AI-specific PM resources: LLM-powered product case studies (how Notion AI, GitHub Copilot, Duolingo Max, and Intercom Fin were built and iterated). Understand the AI PM's unique challenges: non-deterministic systems (AI behavior is not 100% predictable), evaluation-driven development (you need metrics before you build), and the capability-reality gap (AI researchers and engineers often have different intuitions about what works). Write your own definition of what an AI PM does.

Day 2 — Technical Literacy for AI PMs

You don't need to write code, but you must understand: what LLMs are and aren't good at, what RAG is and when it beats fine-tuning, what agents can and can't do reliably, what accuracy/recall/precision mean, how latency and cost affect user experience, and what "hallucination" means for your product's trust model. Spend this day going deep on the technical concepts that most directly affect product decisions.

Day 3 — AI User Research

AI products require different user research techniques. Study: wizard-of-oz testing (simulate AI with a human to validate the experience before building it), expectation calibration research (do users trust the AI too much? too little?), failure mode interviews (what happens when AI is wrong — how do users react?), and longitudinal studies (do users keep using AI products after the novelty wears off?). Design a 5-person user research plan for an AI product of your choice.

Day 4 — AI Feasibility Assessment

Before writing a spec, you must know if the AI can actually do what the product requires. Build an "AI Feasibility Scorecard": rate the task on difficulty (1–5 across dimensions: structured vs unstructured output, tolerance for errors, data availability, latency requirements, and edge case complexity). Validate by running a quick prototype or reviewing research. A PM who ships AI features that don't work loses trust permanently.

Day 5 — Writing AI Product Specs

AI product specs are different from traditional specs. Learn to include: problem statement with user story, AI behavior definition (what should the AI do, in plain English), failure modes and handling (what happens when AI is wrong — fallback, disclosure, correction UX), evaluation criteria (how will we know the AI is good enough?), and guardrails (what should the AI never do or say?). Write a spec for an AI feature in a product you love.

Day 6 — Prototyping with AI

Modern AI PMs can prototype without engineers. Learn: v0.dev (Vercel's AI-powered UI builder), Cursor (AI-assisted coding for non-engineers), Glide and Bubble (no-code with AI integrations), and direct LLM API calls in Jupyter notebooks. Build a clickable prototype of your AI feature. Being able to validate ideas without engineering resources is a superpower.

Day 7 — Sunday Presentation (Week 1)

Present your AI feature spec with a prototype demo. Walk through your feasibility assessment. The community — including engineers — should give feedback on the spec's technical clarity and the product's user value. This cross-disciplinary feedback is gold.

Week 2 — Building (Days 8–14)

Day 8 — AI Metrics & Success Criteria
How do you know your AI feature is working? Define: primary metrics (task completion rate, user satisfaction, feature adoption), AI quality metrics (accuracy, hallucination rate, latency, cost per query), and leading indicators (early signals that predict long-term success). Study how GitHub measured Copilot's success (code acceptance rate, time-to-first-suggestion, lines of code accepted per day). Build a metrics framework for your AI feature.

Day 9 — Evaluation Strategy for Product Teams
Great AI PMs build evaluation discipline into the product development process. Learn: pre-launch evaluation (set quality bars before shipping), A/B testing for AI (how to run experiments on AI features when behavior is non-deterministic), and user feedback loops (thumbs up/down, correction mechanisms). Design an evaluation strategy for your AI feature that includes both automated metrics and user feedback collection.

Day 10 — The AI Product Lifecycle
AI products have a different lifecycle than traditional software. Phase 1: Research (can AI do this at all?). Phase 2: Prototype (does the AI do it well enough to be useful?). Phase 3: Controlled Launch (limited users, gather real-world data). Phase 4: Scale (optimize cost, latency, quality). Phase 5: Iteration (improve model, prompts, or UX based on data). Map an existing AI product through this lifecycle and identify what decisions were made at each phase.

Day 11 — Ethical AI Product Decisions
AI PMs make ethical decisions constantly: Who does the AI serve? Whose data is used to train it? What happens when it fails? Who bears the risk? Study AI ethics frameworks: fairness and bias (measure demographic performance gaps), transparency (should users know they're talking to AI?), accountability (who is responsible when AI causes harm?), and safety (what could go wrong, and how bad is it?). Write an ethical risk assessment for your AI feature.

Day 12 — Building with AI APIs: A PM's Practical Guide
AI PMs should be able to run experiments themselves. Learn: how to call OpenAI and Anthropic APIs from a Jupyter notebook, how to measure cost and latency, how to run 50 test cases against two different prompts, and how to interpret the results. Build a simple prompt comparison experiment for your AI feature and present the findings as a product decision document.

Day 13 — Stakeholder Communication
AI PMs must translate between three worlds: research (what's possible), engineering (what's buildable), and business (what's valuable). Practice: writing an AI feature proposal for a non-technical executive (focus on user value and business metrics), briefing an engineering team on user research findings (focus on what matters for model behavior), and presenting evaluation results to leadership (focus on confidence levels and risks). Write all three versions of one communication.

Day 14 — Sunday Presentation (Week 2)
Present your AI feature proposal in three versions: one for a CEO, one for an engineer, and one for a user researcher. The community will evaluate how well you translated the same information for different audiences. This is the core skill of product management.

Week 3 — Deepening (Days 15–21)

Day 15 — Case Study: GitHub Copilot
Deep-dive on the most successful AI product ever: GitHub Copilot. Read everything public: the GitHub blog posts on Copilot's development, Nat Friedman's interviews, the research papers on code generation, the feature evolution from Copilot → Copilot Chat → Copilot Workspace. Answer: What was the initial product hypothesis? What metrics did they use? What was the biggest user research surprise? What UX patterns did they invent? Write a 2-page product analysis.

Day 16 — AI Product Pricing Strategy
AI products have unusual economics: cost is variable (you pay per LLM token), quality improves over time, and usage patterns are unpredictable. Study: per-seat pricing (Copilot — $19/month regardless of usage), usage-based pricing (OpenAI API — pay per token), freemium (Notion AI — limited free tier, paid for power use), and outcome-based pricing (pay per successful task completion, emerging model). Build a pricing model for your AI feature.

Day 17 — Building the AI PM Portfolio
AI PMs need a portfolio just like engineers. Build: a product teardown (pick any AI product and write a detailed analysis of its design decisions and metrics strategy), a product spec (for an AI feature you want to exist), a user research report (interview 5 people about how they use AI tools), and a competitive analysis (map the AI product landscape in one vertical). Host all of this on Notion, and link it from your GitHub.

Day 18 — Contribution Day: Documentation & Community
AI PMs contribute differently than engineers. Write thorough documentation for an open-source AI project that has poor docs. Translate a technical paper into a product-readable summary. Contribute to the DEEPSEED community by writing a guide on "How to evaluate AI product ideas before building them." Your written thinking is your contribution.

Day 19 — AI Product Trends: What's Coming
Study the emerging trends that will shape AI products in the next 2 years: AI agents in the browser and desktop (computer use), multimodal voice interfaces, personalized AI (models that adapt to you over time), AI-native data products (AI that manages and queries your own data), and AI in physical systems (robotics, IoT). Write a product brief for one product that doesn't exist yet but should — based on current AI capabilities.

Day 20 — Launch Planning
Plan a launch for your AI feature: pre-launch checklist (evaluation passed? safety review done? rollout plan defined? fallback plan ready? support team trained?), launch communication (internal announcement, external blog post, product changelog), and staged rollout (10% → 25% → 50% → 100% with metric gates). Write the full launch plan for your AI feature. Good launches have plans for when things go wrong.

Day 21 — Sunday Presentation (Week 3)
Present your GitHub Copilot analysis. The community votes on the most insightful observation. Then pitch your "product that doesn't exist yet" — engineers will tell you if it's technically feasible, and PMs will poke holes in the business model. This is what product reviews at real companies feel like.

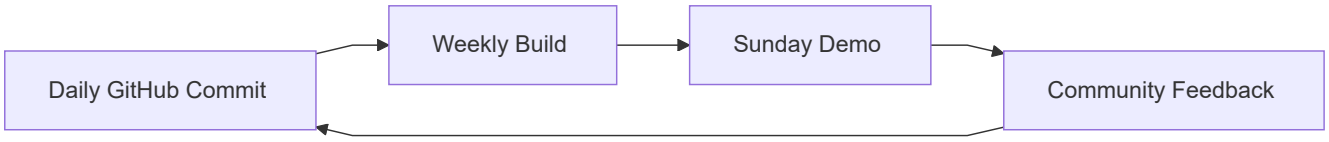
Week 4 — Capstone (Days 22–30)

Build a complete AI product document for a real AI product you want to exist: user research synthesis (interviews with 5+ potential users), full product spec with AI behavior definition and failure handling, feasibility assessment based on current capabilities, evaluation framework with metrics and test plan, pricing strategy, launch plan, and an ethical risk assessment. Present this as a board-level product proposal. This document is your AI PM portfolio piece.

Top Open-Source Projects to Contribute To

While AI PMs don't always write code, they can contribute enormously to: **Hugging Face Hub** (write model cards for models that lack them, document datasets), **LangChain documentation** (`langchain-ai/langchain` — clear docs have massive impact), and **OpenAssistant** (`LAION-AI/Open-Assistant` — the dataset and evaluation effort needs non-engineers deeply). Write product analyses of open-source tools and share them publicly — these become reference resources for the community and demonstrate your thinking.

Community Rules & Culture



The DEEPSEED community operates on four cultural principles that must be lived, not just stated.

Build in public. Every day of work goes to GitHub. Not just the successes — the failures, the half-working scripts, the confused `learnings.md` entries. Struggle documented publicly is more valuable than success kept private, because it helps the next person who gets stuck in the same place.

Sunday presentations are non-negotiable. Presenting forces clarity of understanding in a way that private study never does. You cannot demo something you don't understand. Every Sunday presentation should have: a live demo (not slides), an honest account of what didn't work, and at least one question for the community.

Contribute to what you consume. Every week, spend at least 30 minutes on an open-source project in your field. It doesn't have to be a PR — it can be a detailed issue report, a documentation clarification, or a well-researched response to someone's GitHub issue. Open-source is how this whole field exists.

Teach as you learn. Write a short post (in your `learnings.md`) every day. At the end of 30 days, you will have 30 pieces of writing that explain your field clearly. These are the beginning of your technical blog, your documentation contributions, and your community legacy within DEEPSEED.

Final Note

The 30-day challenge is not about becoming a world expert in 30 days. It is about proving to yourself — and to the community — that you can go deeper than a bootcamp, work consistently without external deadlines, build real things under time pressure, and contribute to the community that is investing in you.

The person who finishes this challenge with a GitHub full of daily commits, a deployed capstone project, and four Sunday presentations has done something that most engineers — with or without AI — never do. They have built a habit of building.

Welcome to DEEPSEED. Now go build.

Document written by Fonyuy Gita
DEEPSEED AI Community — 30-Day AI Mastery Challenge
"Deep roots, strong seeds."